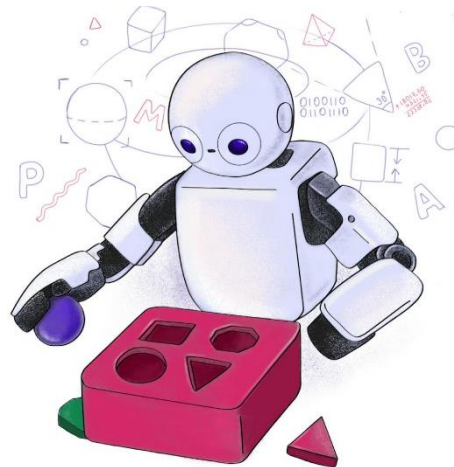
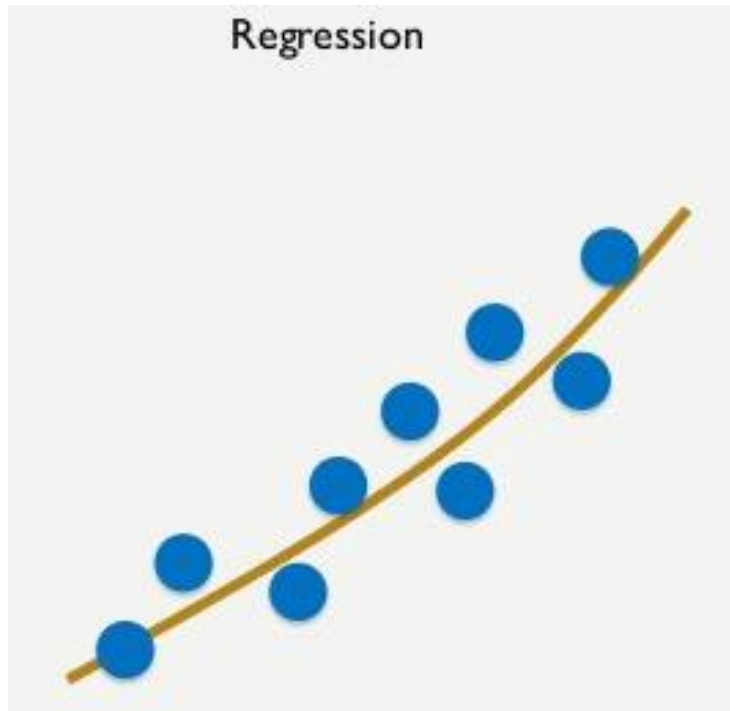


C24 – Inteligência Artificial: *Regressão*



Regressão



$h(x)$ aproxima o comportamento geral dos dados.

- Tarefa (ou problema) de **aprendizado supervisionado**.
 - Os valores de entradas (atributos) e saídas esperadas (rótulos) são conhecidos.
- Envolve encontrar uma função, $h(x)$, que **mapeie** os atributos em **valores reais**.

Motivação

- **Exemplo 1:** Estimar o preço de casas.

5 quartos, 3 vagas na garagem, 1 piscina, cond. de luxo, 500 m^2



R\$ 1.000.000,00

1 quarto, sem vaga na garagem, bairro afastado, 70 m^2



R\$ 200.000,00

2 quartos, 1 vaga na garagem, centro, 200 m^2

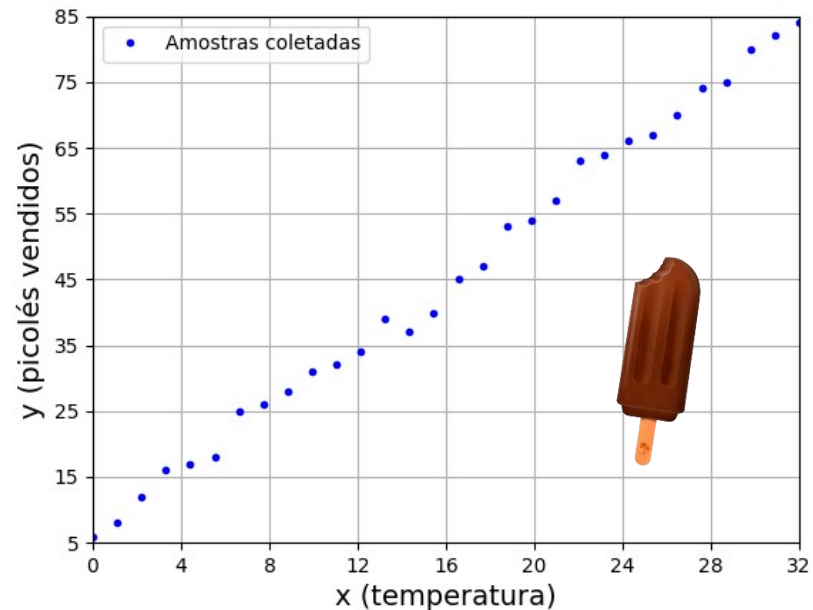


???

- Podemos usar **regressão** para encontrar uma **relação matemática**, $h(x)$, entre o n° de quartos, n° de vagas na garagem, se tem piscina, localização, área, etc. de uma casa e seu valor.
- **Objetivo:** agilizar o processo de avaliação de propriedades.

Motivação

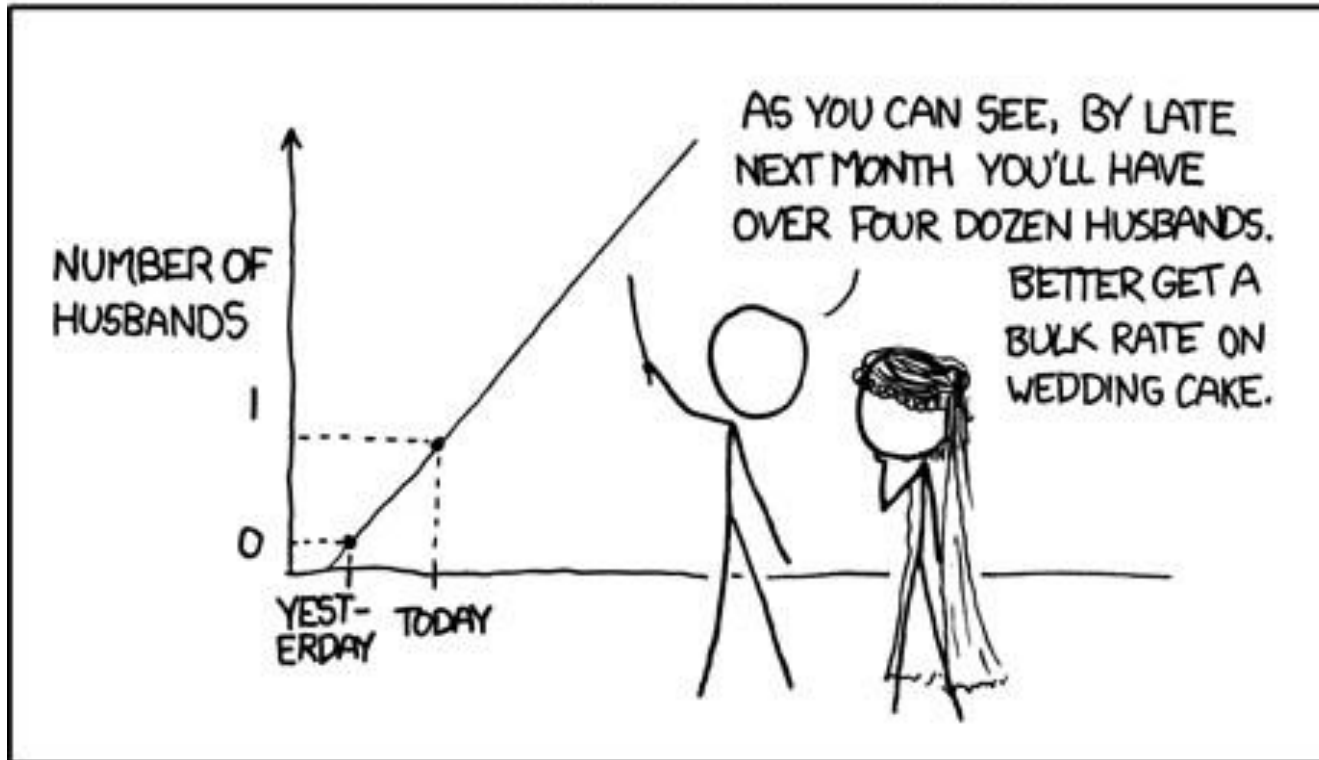
- **Exemplo 2:** Estimar as vendas de picolés.



- Podemos usar regressão para encontrar um **mapeamento**, $h(x)$, entre a temperatura média de um dia e a quantidade de picolés vendidos.
- **Objetivo:** reduzir o desperdício e aumentar os lucros.

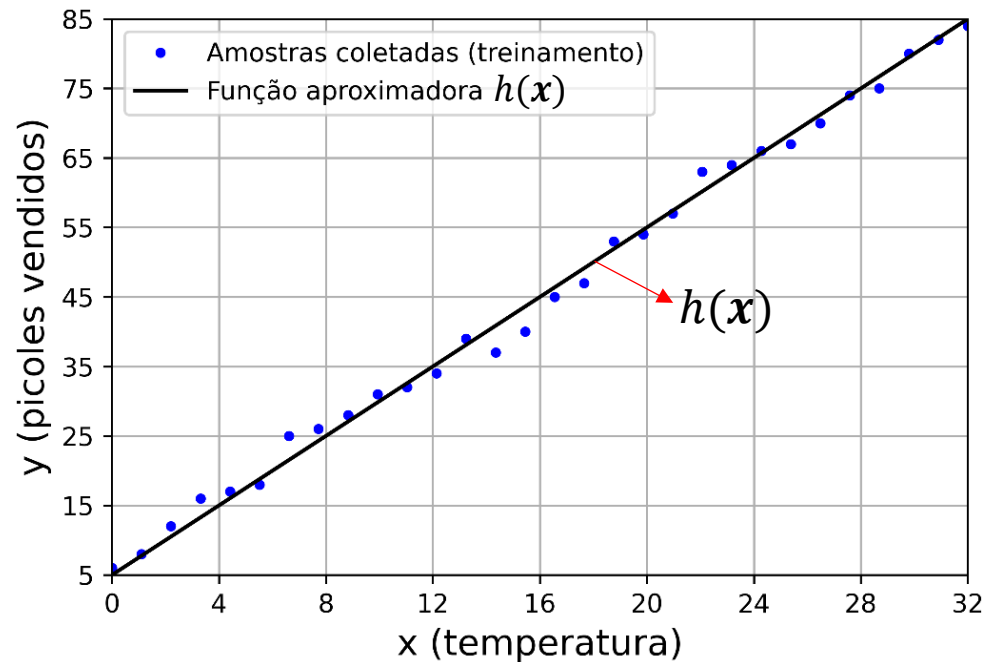
Como podemos encontrar as funções de mapeamento, $h(\boldsymbol{x})$?

MY HOBBY: EXTRAPOLATING



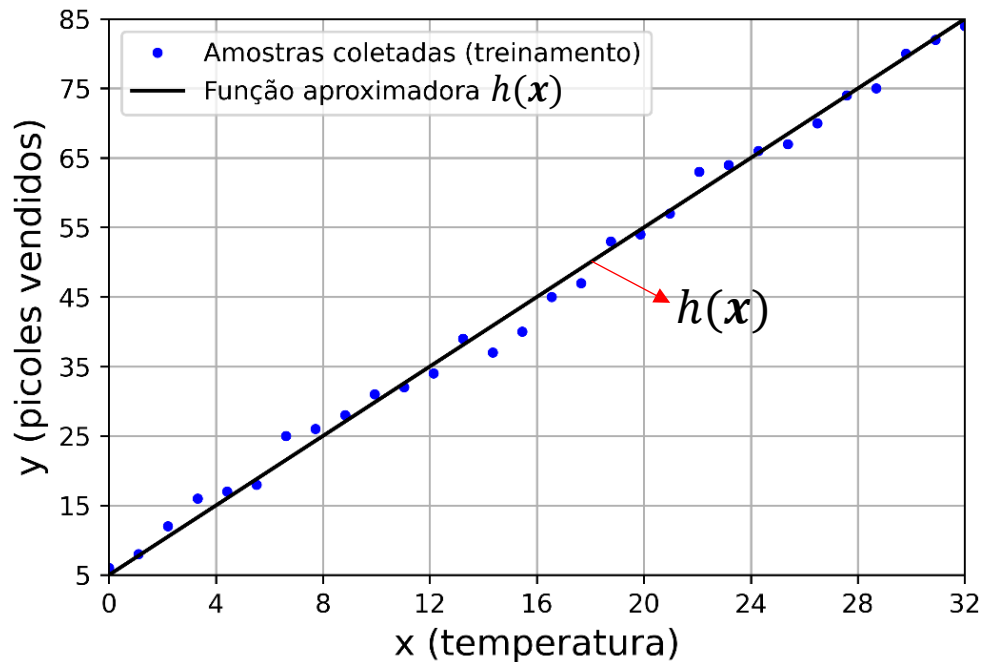
Usando
regressão linear!

Regressão linear



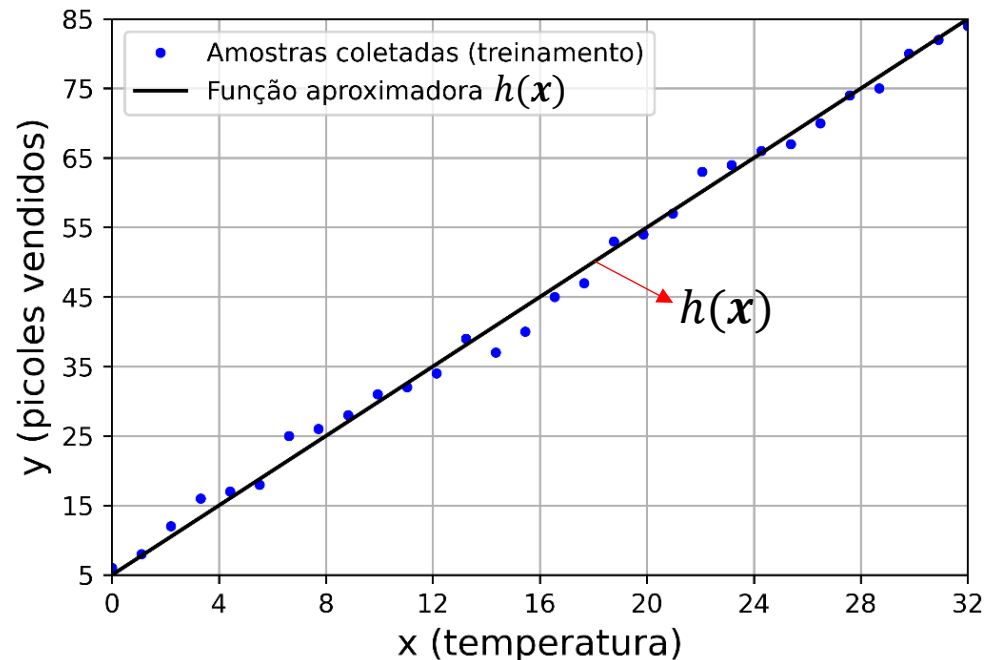
- É um dos algoritmos mais simples, antigos e usados de aprendizado de máquina.
- Nos dará **intuições** importantes para o **entendimento** de **algoritmos mais complexos**, como **classificadores** e **redes neurais**.

Qual o objetivo da regressão linear?



- **Objetivo:** encontrar uma **função aproximadora**, $h(x)$, que **mapeie** os **atributos**, x , em uma variável de saída, $\hat{y} = h(x)$, de tal forma que $h(x)$ se ajuste aos dados coletados de **forma ótima**.
- **Ótimo** no sentido da **minimização da diferença** entre as previsões feitas por $h(x)$, (i.e., os valores de $\hat{y}$), e os **rótulos**, (i.e., os valores esperados de saída, y).

Função hipótese



- No contexto da regressão, a **função aproximadora**, $h(x)$, é chamada de **função hipótese**, pois refere-se à **explicação proposta** para a **relação** entre as entradas, x e o rótulo, y .
- Por exemplo, anteriormente, **hipotetizamos** que uma **reta** explicaria bem a **relação** entre a **temperatura média** de um dia e o número de **picolés vendidos**.

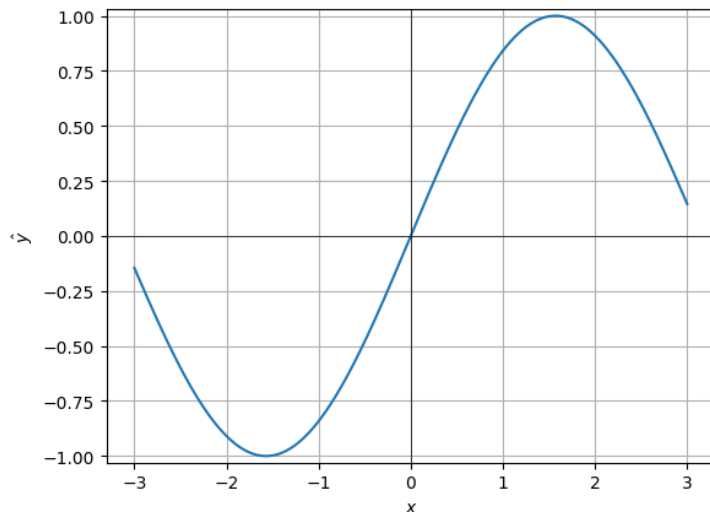
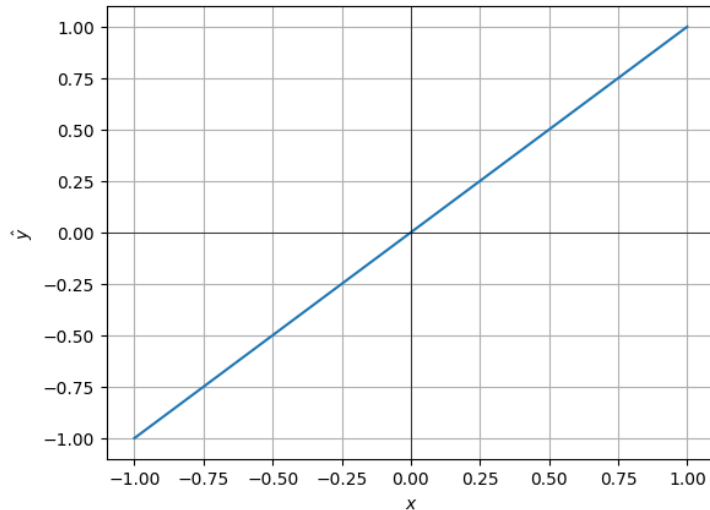
Por que linear?

- Porque a saída da **função hipótese**, $\hat{y} = h(x)$, é **modelada** como sendo uma **combinação linear dos atributos**, x , **em relação aos pesos**.
- Em geral, usamos **polinômios** para modelar essa combinação.
 - $\hat{y} = h(x) = a_0 + a_1x$ (polinômio de grau 1, reta, com uma variável).
 - $\hat{y} = h(x) = a_0 + a_1x_1 + a_2x_2$ (polinômio de grau 1, plano, com duas variáveis).
 - $\hat{y} = h(x) = a_0 + a_1x + a_2x^2$ (polinômio de grau 2 com uma variável).
 - $\hat{y} = h(x) = a_0 + a_1x_1 + a_2x_1^2x_2$ (polinômio de grau 3, com duas variáveis).

Por que polinômios?

- Pois segundo o teorema da aproximação de **Stone-Weierstrass** “qualquer função contínua no intervalo fechado $[a, b]$ pode ser uniformemente aproximada por um **polinômio**”.
- Portanto, polinômios podem modelar qualquer tipo de mapeamento (lineares ou não lineares) entre os atributos, $\mathbf{x}$, e $\hat{y}$, bastando apenas **encontrar os pesos, $a_k \forall k$ e o grau ideal.**

Por que polinômios?



- Podemos modelar um **mapeamento linear** com a equação de uma reta:

$$\hat{y} = h(x) = a_0 + a_1x.$$

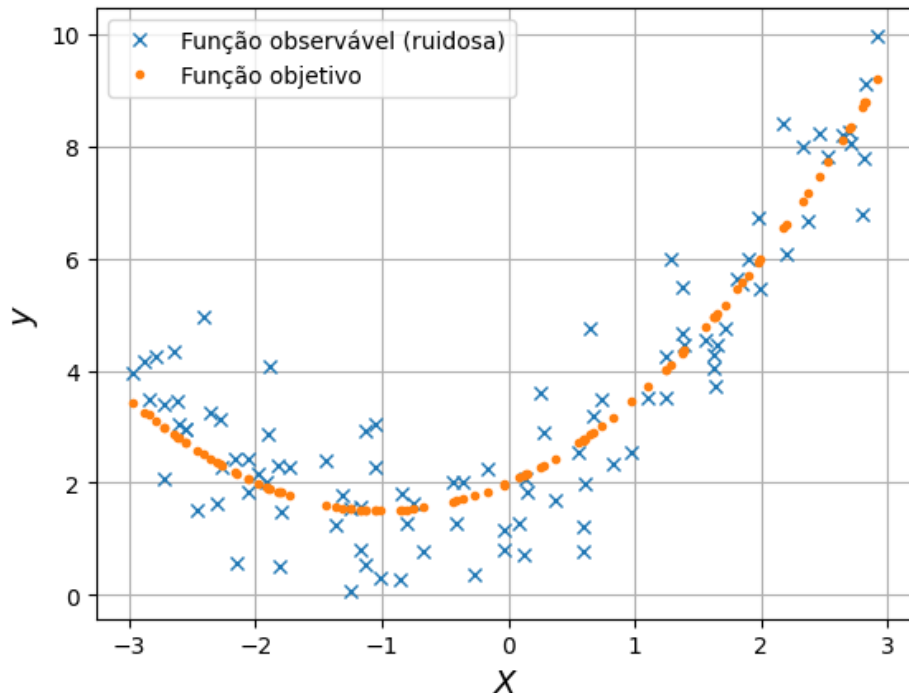
- Podemos ter um **mapeamento não-linear** com uma equação que aproxima uma senoide:

$$\hat{y} = h(x) = a_0 + a_1x + a_2x^3 + a_3x^5 + a_4x^7 + a_5x^9.$$

- Também podemos ter mapeamentos **lineares e não lineares com dois ou mais atributos**.

$$\begin{aligned} \hat{y} &= h(x_1, x_2) = a_0 + a_1x_1 + a_2x_2 + a_3x_3 + a_4x_1x_3 \\ &+ a_5x_1^3 + a_6x_1x_2x_3^2. \end{aligned}$$

A flexibilidade dos polinômios



- Grau 1: linha reta
- Grau 2: parábola com um ponto de inflexão,
- Grau 3: curva com até dois pontos de inflexão
- Graus maiores: curvas **cada vez mais flexíveis**
- **Observações:**
 - Maior grau $\neq$ melhor modelo.
 - Flexibilidade excessiva pode levar ao *overfitting*.
 - Por outro lado, flexibilidade de menos pode levar ao *underfitting*.

Como encontramos os pesos e o
melhor grau?

Objetivo da regressão linear

- Por enquanto, vamos assumir que já conhecemos o melhor grau do polinômio.
- Como temos os atributos e os rótulos, o **objetivo** passa a ser encontrar os **pesos (ótimos)** que **minimizam a distância (i.e., o erro)** entre **predição**, $\hat{y}$ (i.e., saída do modelo) e os **rótulos**, y .
- Escrevendo como um problema de otimização, temos que o **objetivo** é
 - Encontrar o **vetor de pesos**, $\mathbf{a} = [a_0 \dots a_K]^T \in \mathbb{R}^{K+1 \times 1}$, que minimize o **erro**, dado por uma **função de erro**, $J_e(\mathbf{a})$, entre a aproximação, $\hat{y}$, e o valor esperado, y , **para todos os exemplos do conjunto de treinamento**.

$$\mathbf{a}_{\text{ótimo}} = \min_{\mathbf{a}} J_e(\mathbf{a}).$$

Para treinarmos o modelo,
precisamos de uma função de erro

Função de erro

- Existem várias possibilidades para a **função de erro**. Porém, em problemas de regressão, é comum utilizar-se o **erro quadrático médio (EQM)**

$$J_e(\mathbf{a}) = \frac{1}{N} \sum_{n=0}^{N-1} (y(n) - \hat{y}(n))^2 = \frac{1}{N} \underbrace{\left\| \mathbf{y} - \underbrace{\mathbf{X}\mathbf{a}}_{\hat{\mathbf{y}}} \right\|^2}_{\text{forma matricial}},$$

onde N é o número total de amostras, n é o número da amostra, $\mathbf{y} = [y(0) \dots y(N-1)]^T \in \mathbb{R}^{N \times 1}$ e $\hat{\mathbf{y}} = [\hat{y}(0) \dots \hat{y}(N-1)]^T \in \mathbb{R}^{N \times 1}$ são **vetores** contendo todos os N valores esperados e de predições, respectivamente e $\mathbf{X} = [\mathbf{x}(0) \dots \mathbf{x}(N-1)]^T \in \mathbb{R}^{N \times K+1}$ é uma **matriz** contendo todos os N vetores de atributo.

- O **EQM** nada mais é do que a **média aritmética do quadrado dos erros**.

Minimização da função de erro

- Então, para encontrarmos o **vetor de pesos ótimo**, $\mathbf{a}_{\text{ótimo}}$, devemos **minimizar a função de erro**

$$\mathbf{a}_{\text{ótimo}} = \min_{\mathbf{a}} \frac{1}{N} \left\| \mathbf{y} - \underbrace{\mathbf{X}\mathbf{a}}_{\hat{\mathbf{y}}} \right\|^2.$$

- Por ser constante, o termo $1/N$ não influencia na minimização e, portanto, pode ser omitido na equação acima.

Como encontramos os pesos que minimizam a função de erro?

Como encontrar os pesos ótimos?

- Existem duas formas principais para se encontrar o **conjunto de pesos ótimo**

- **Equação normal**

- Também conhecido como método dos mínimos quadrados ordinários.
- É uma solução analítica.
- É uma abordagem puramente algébrica, resultando em uma equação fechada.

$$\mathbf{a}_{\text{ótimo}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- Encontra a solução ótima de uma única vez.

- **Gradiente descendente**

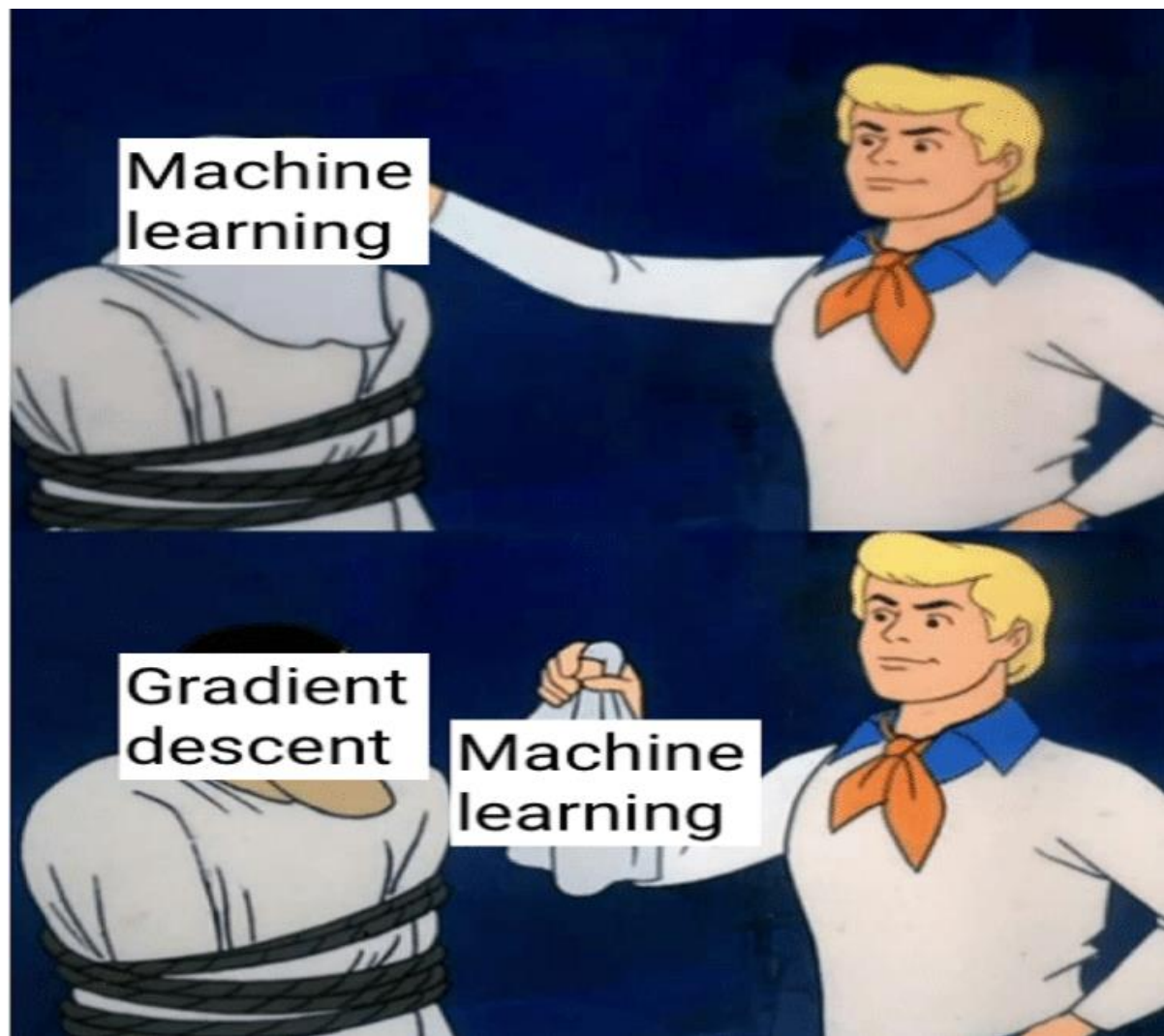
- É uma otimização iterativa.
- O modelo aprende os pesos "passo a passo".
- Pode não encontrar a solução ótima, devido a vários fatores (e.g., passo de aprendizagem, sobreajuste/subajuste, número de iterações, etc.)

Comparativo das abordagens

Característica	Gradiente descendente	Equação normal
Tipo	Algorítmico/Iterativo	Analítico/Direto
Complexidade	$O(IK^2)^*$	$O(K^3)^{**}$
Escalabilidade	Alta (lida bem com milhões de dados)	Baixa (lenta para muitas variáveis)
Parâmetros	Exige ajuste de α	Nenhum
Uso principal	Regressão linear e logística/Redes neurais/SVM	Estatística clássica/Regressão linear/ <i>Datasets</i> pequenos
Observações	• Pode exigir muitas iterações para convergir. • A função de erro deve ser diferenciável. • Complexidade computacional ajustável à quantidade de memória e recursos disponíveis.	• Não encontra solução para modelos não lineares (e.g., redes neurais). • Consome muita memória se K e N são muito grandes.

* I é o número de iterações

** Complexidade para o cálculo da inversa de $X^T X$

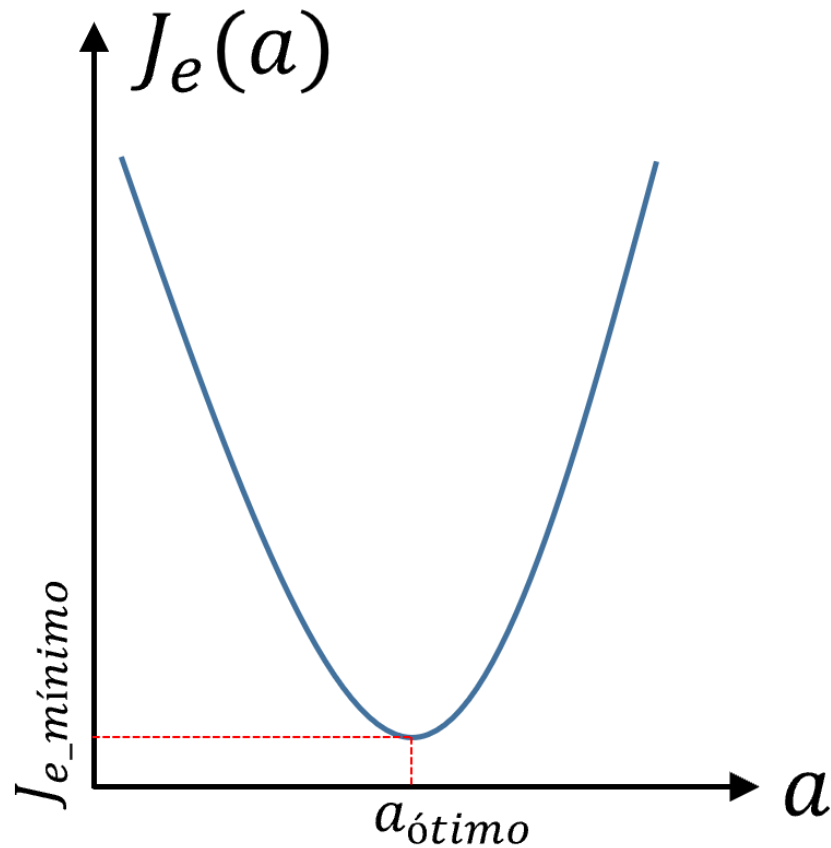


O gradiente descendente
está por trás dos
algoritmos mais populares
de ML.

Regressão linear
Regressão logística
Redes neurais *multilayer perceptron*
Deep learning (CNNs, RNNs,
Transformers)
Máquinas de vetores de suporte (SVM)

Como o gradiente descendente é a abordagem mais usada com grande parte dos algoritmos de ML, vamos focar no seu aprendizado.

Superfície de erro



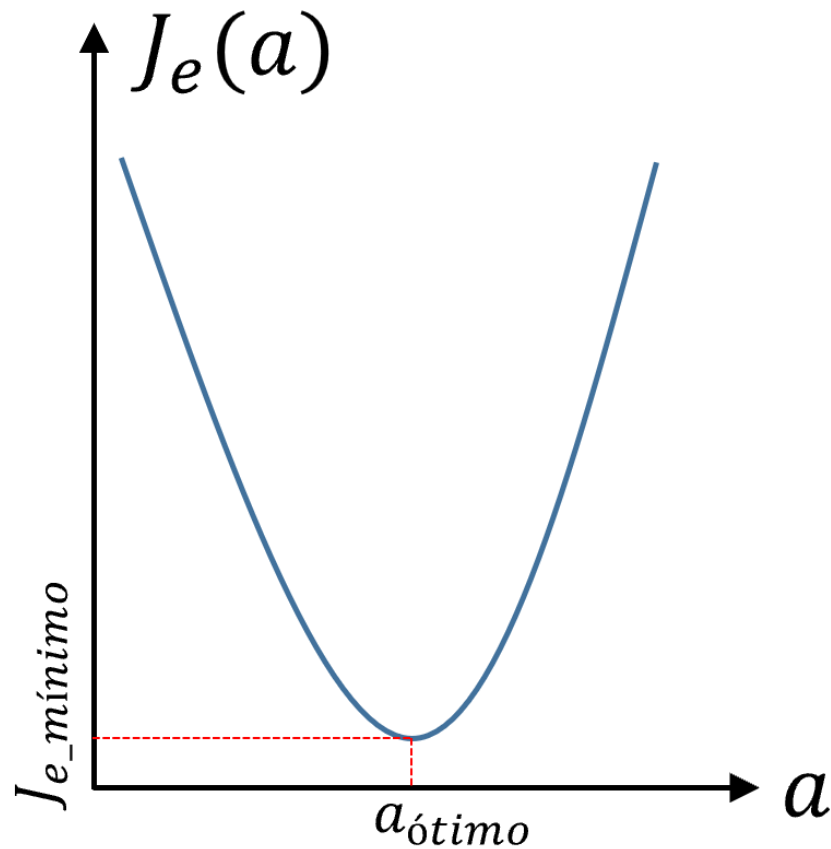
- Antes de discutirmos como o gradiente descendente funciona, precisamos entender o que é a **superfície de erro**.
- Ela é uma **representação geométrica da função de erro**.
- Ela mostra o quão "errado" o nosso modelo está **para cada combinação possível de pesos**

$$J_e(\mathbf{a}) = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{a}\|^2$$

- A plotamos variando os pesos e anotando os respectivos erros.

OBS.: Superfície de erro para um função hipótese com um atributo.

Superfície de erro



- O ponto mais baixo da superfície nos dá vetor de **pesos ótimos**, $a_{\text{ótimo}}$.

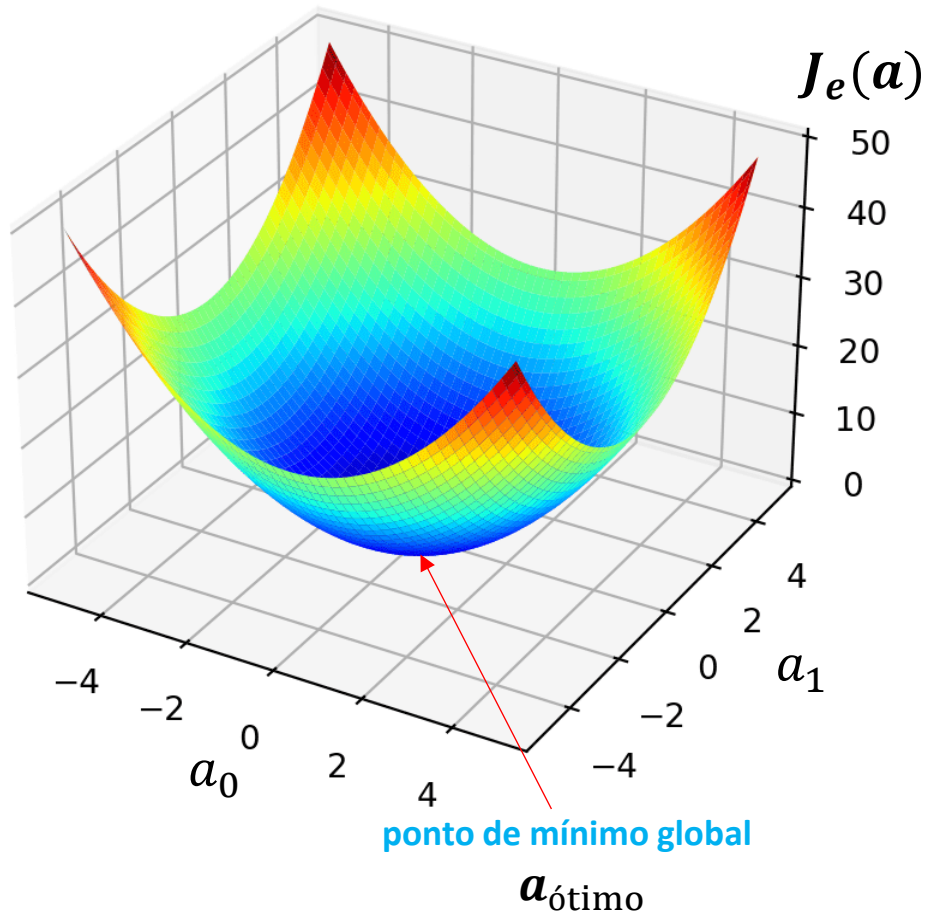
- Devido ao EQM ter uma forma quadrática,

$$\frac{1}{N} \|y - Xa\|^2$$
$$= \frac{1}{N} \left(y^T y - y^T Xa^T - a^T X^T y + \underbrace{a^T X^T Xa}_{\|Xa\|^2 \geq 0} \right),$$

sua superfície de erro **sempre terá o formato de uma parábola (ou tigela)**.

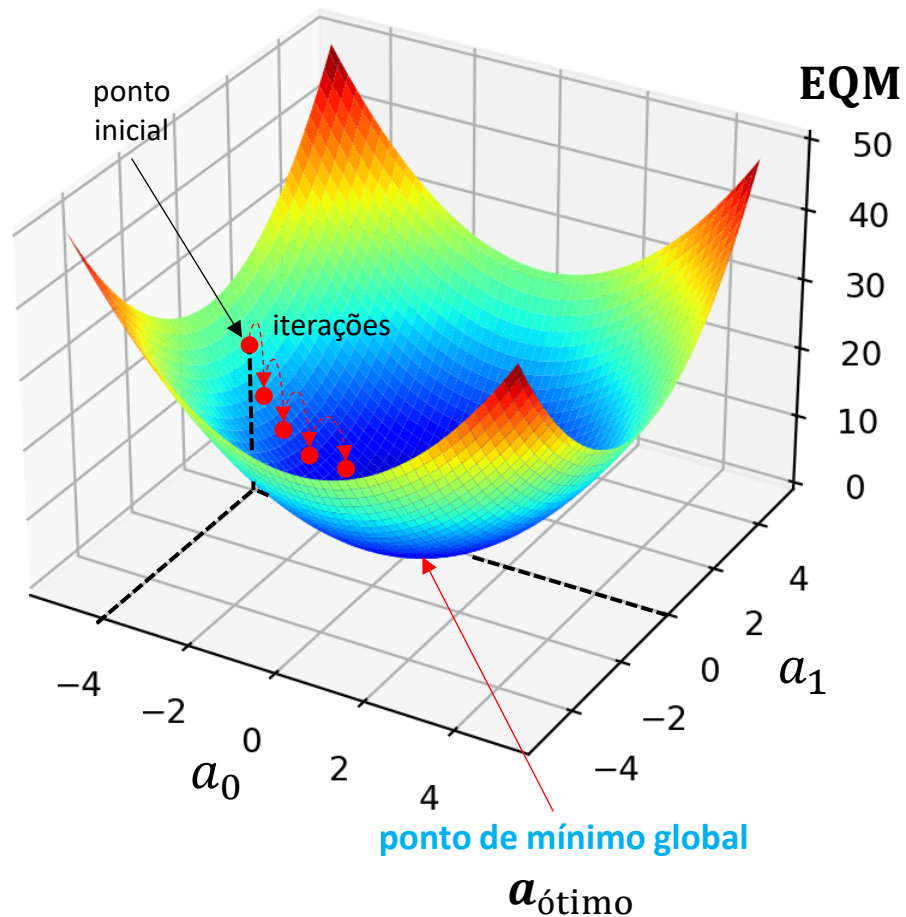
- Além disso, devido ao termo quadrático ser positivo, ela terá a **concavidade sempre voltada para cima**.

Superfície de erro



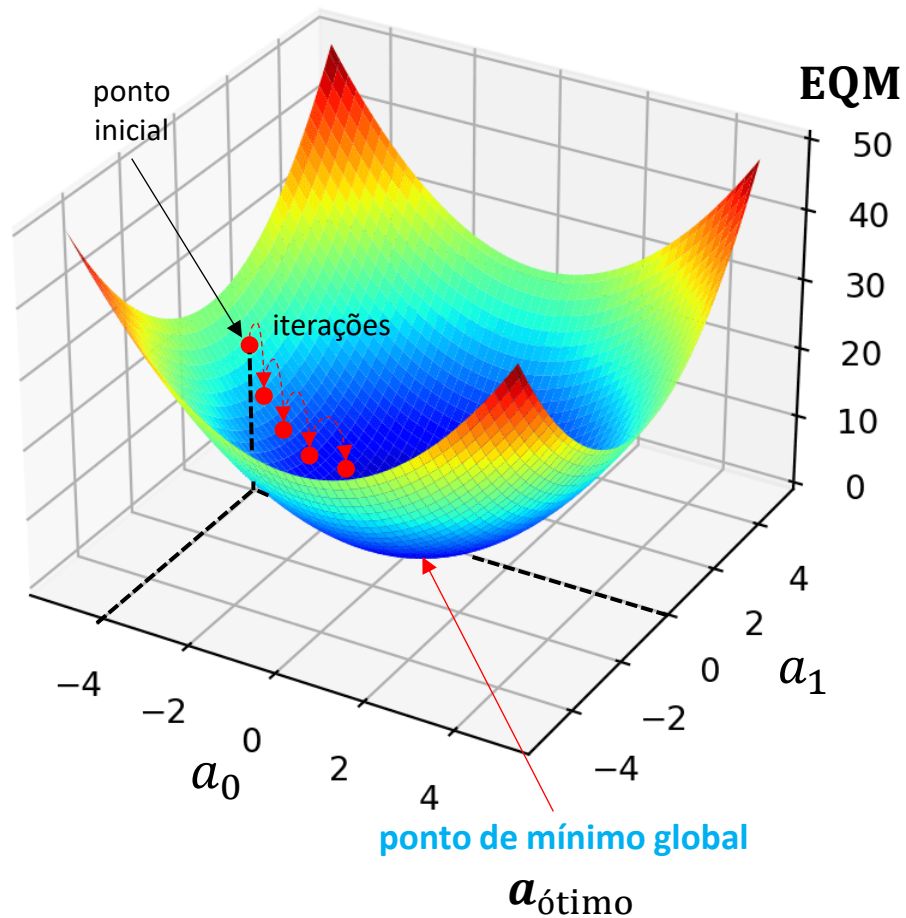
- O que isso significa na prática?
 - Existe apenas **um único** ponto mais baixo (chamado de **mínimo global**).
 - Portanto, basta **seguirmos ladeira abaixo a partir de qualquer ponto** que encontraremos a melhor solução possível, $a_{\text{ótimo}}$.
- Em geral, o **erro mínimo** nunca será igual a 0 devido aos dados estarem corrompidos por algum tipo de ruído.

Gradiente descendente



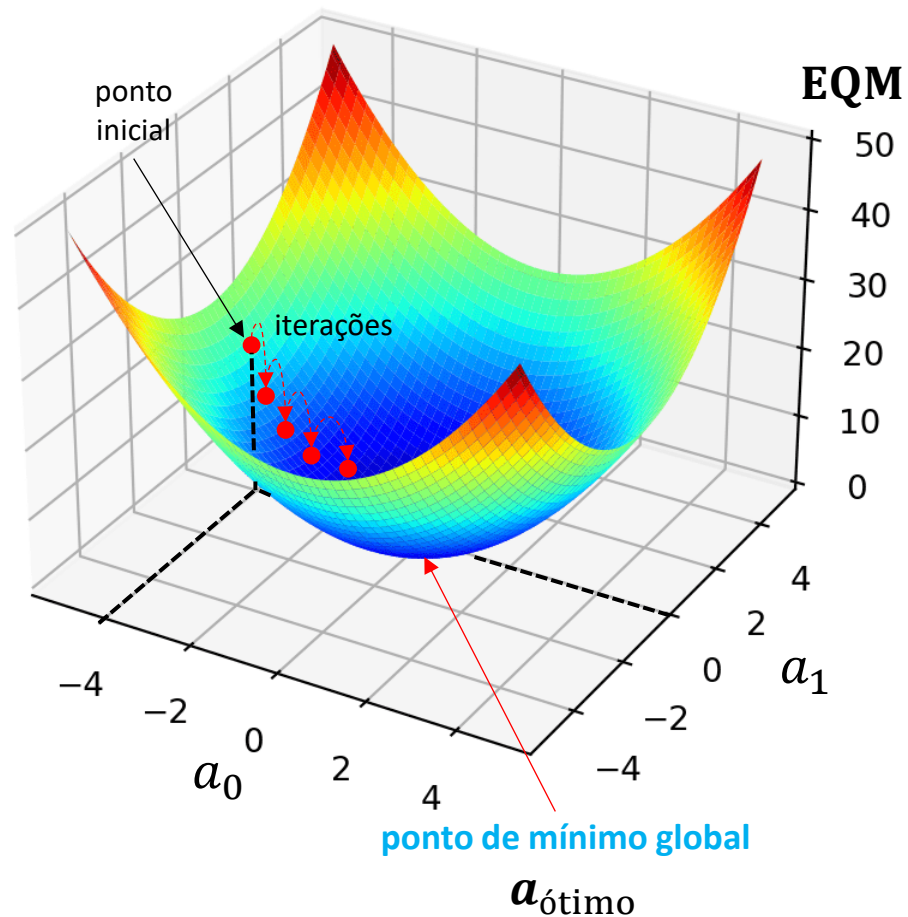
- O **gradiente descendente** procura de forma **iterativa** o ponto mais baixo da superfície de erro.
 - O GD **desce a ladeira** em busca de $a_{\text{ótimo}}$.
- Ele é inicializado com **pesos (i.e., em pontos) aleatórios**.
- E, a cada iteração, ele **refina** os valores dos pesos, ficando mais e mais próximo do ponto de mínimo.

Gradiente descendente



- Esse **processo iterativo de busca** que tem como **objetivo** encontrar o **ponto de mínimo** é chamado de **otimização** e, no contexto de ML, é conhecido como **treinamento do modelo**.
 - Treinamento é o processo de **atualizar o modelo (i.e., os pesos)** para obter previsões melhores a cada **iteração**.

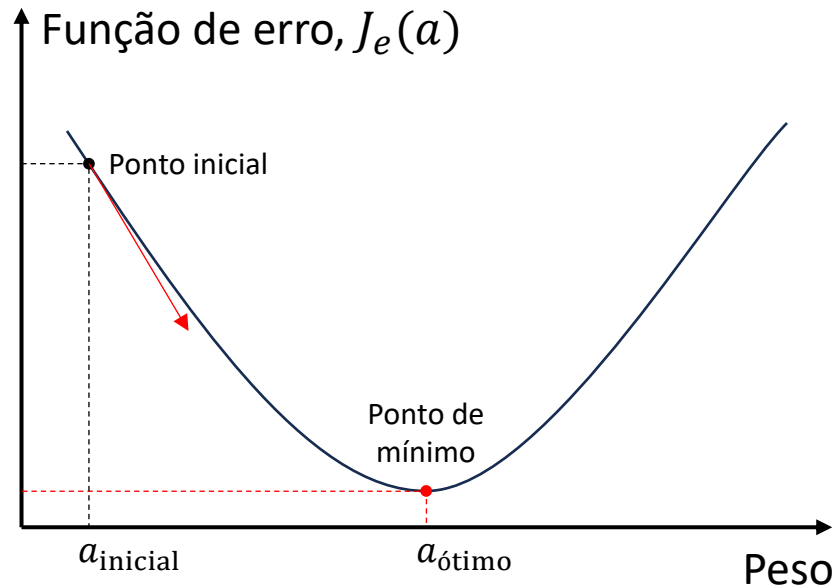
Gradiente descendente



- O **treinamento** do modelo se **baseia** em **informação trazida pela da função de erro**.
- Essa **informação aponta** para a **direção do ponto de mínimo**.
- Como veremos, com essa **informação**, o **gradiente descendente caminha iterativamente** através da **superfície de erro** até o **ponto de mínimo**.

Como funciona o gradiente descendente?

Vetor gradiente

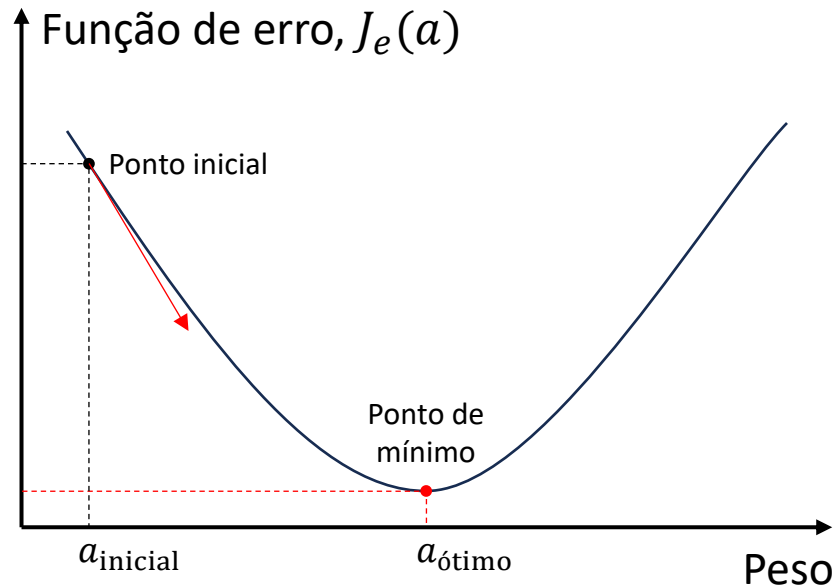


- Se nós **diferenciarmos a função de erro em relação aos pesos**, nós obtemos o **vetor gradiente**

$$\nabla J_e(\mathbf{a}) = \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}.$$

- Ele sempre aponta na **direção de maior taxa de crescimento da função** a partir do ponto, $\mathbf{a}$, onde ele é calculado.

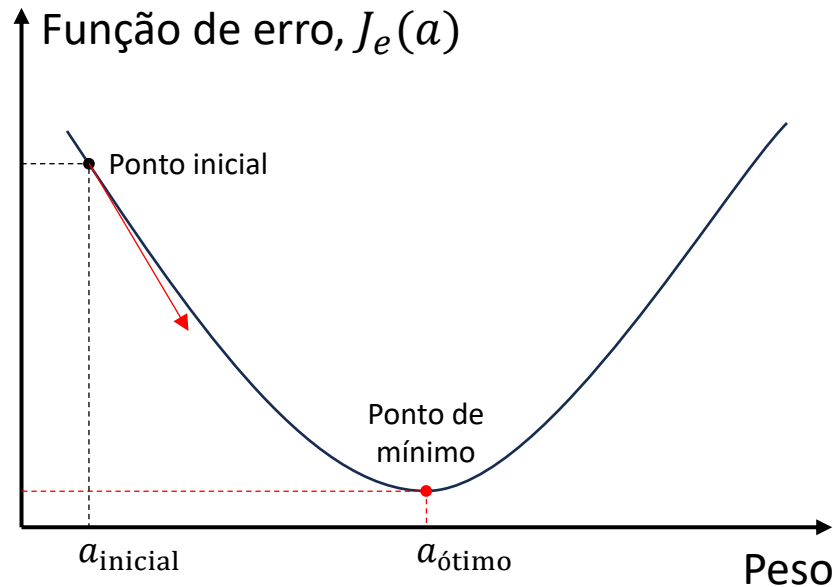
Vetor gradiente



No caso da função hipótese ter apenas um peso, o vetor gradiente dá a inclinação de uma **reta tangente** ao ponto onde ele é calculado.

- O gradiente pode ser também interpretado como a **inclinação** de um **hiperplano tangente à função** no ponto onde ele é calculado.
 - Quanto **maior o valor absoluto** do gradiente, **mais inclinado** é o **hiperplano tangente** àquele ponto.
 - Portanto, **um valor igual a 0** indica **inclinação nula**.
 - Onde isso ocorre?
 - **Resposta:** Em pontos de máximo e de mínimo.

Vetor gradiente



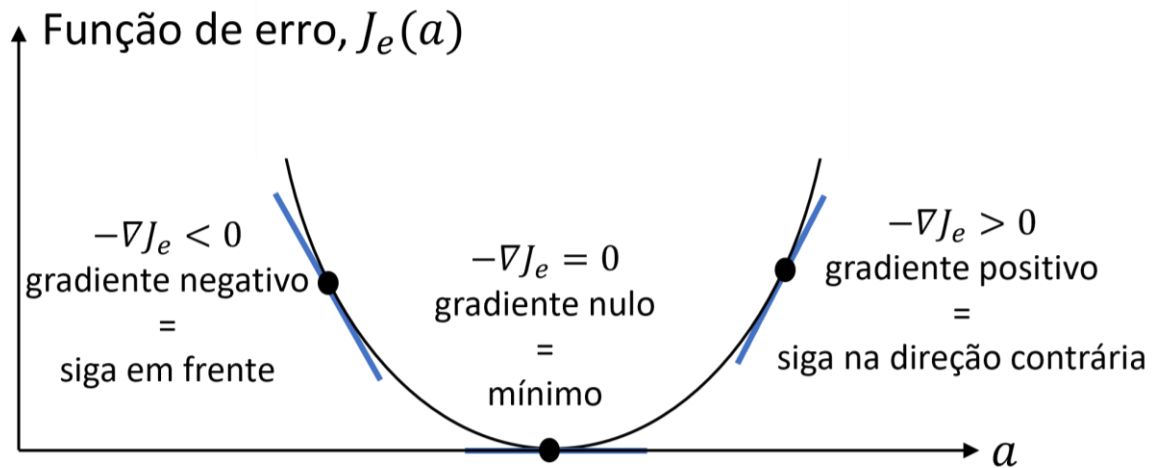
O objetivo é minimizar a função de erro indo na direção contrária à indicada pelo gradiente.

- Porém, queremos **encontrar o mínimo** da função, o que devemos fazer?
- Basta irmos na **direção oposta** à indicada pelo vetor gradiente (i.e., negativo do gradiente)

$$-\nabla J_e(\mathbf{a}) = -\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}$$

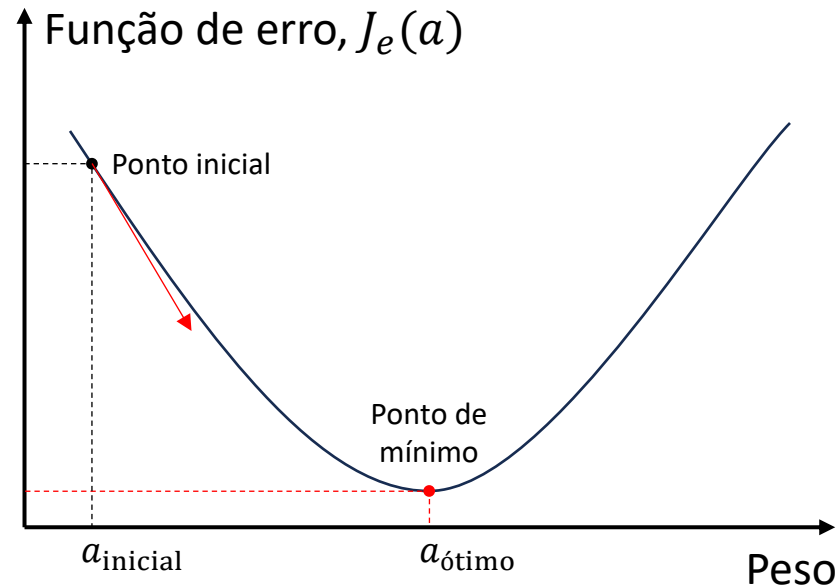
- O **negativo do vetor gradiente** aponta para a **direção de maior taxa de decrescimento da função** a partir do ponto onde ele é calculado.

Vetor gradiente



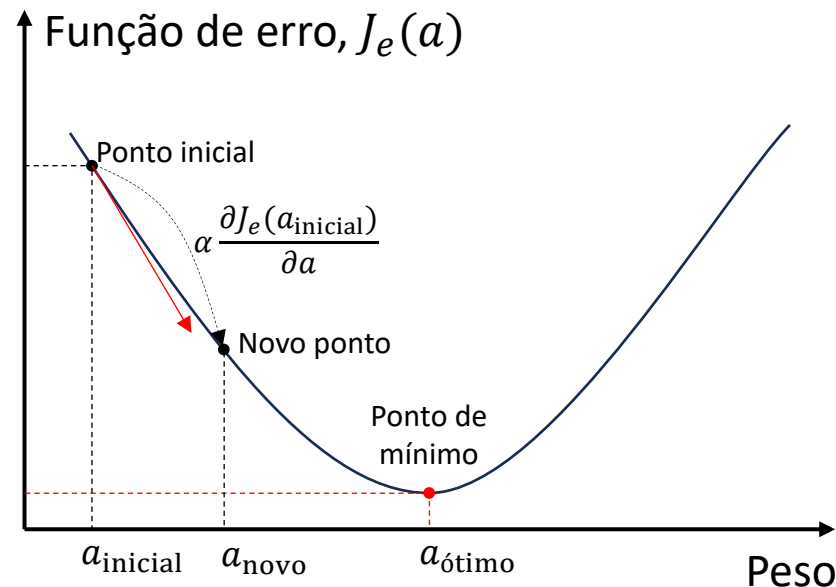
- Quando seguimos na **direção contrária a da máxima taxa de crescimento**, estamos indo na **direção de decrescimento mais rápido da função**.
- Portanto, um elemento do vetor gradiente com sinal:
 - - (reta com inclinação negativa) indica que o **ponto de mínimo está à frente**.
 - + (reta com inclinação positiva) indica que o **ponto de mínimo está atrás**.
 - 0 (reta com inclinação nula) indica que **ponto de mínimo foi encontrado**.

Qual a distância até o mínimo?



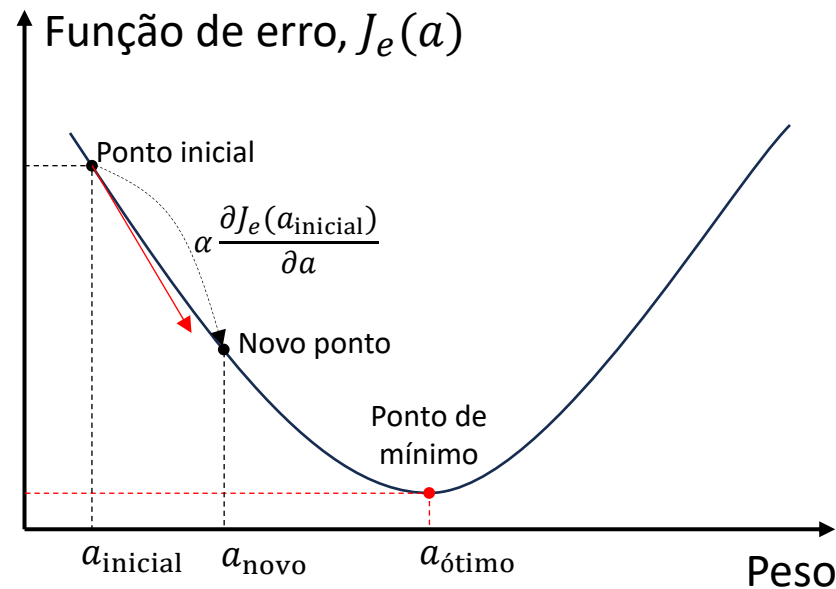
- Entretanto, o gradiente **não dá informação sobre a distância até o ponto de mínimo**, mas pelo menos sabemos a direção correta.
 - Gradiente fornece **apenas** a direção e a taxa de crescimento.
- Podemos fazer a analogia com uma bola solta em uma ladeira.
- A gravidade dá a direção até a parte mais baixa da ladeira, mas não dá a distância até lá.

Passo de aprendizagem



- Portanto, se quisermos **seguir até o ponto de mínimo** a partir de um ponto qualquer, podemos **dar um passo na direção oposta à apontada pelo gradiente**.
- Nós sabemos a direção do ponto de mínimo, então basta **escolhermos o tamanho do passo** para darmos naquela direção.
- O tamanho do passo é definido por α , chamado de **passo de aprendizagem**.

Passo de aprendizagem

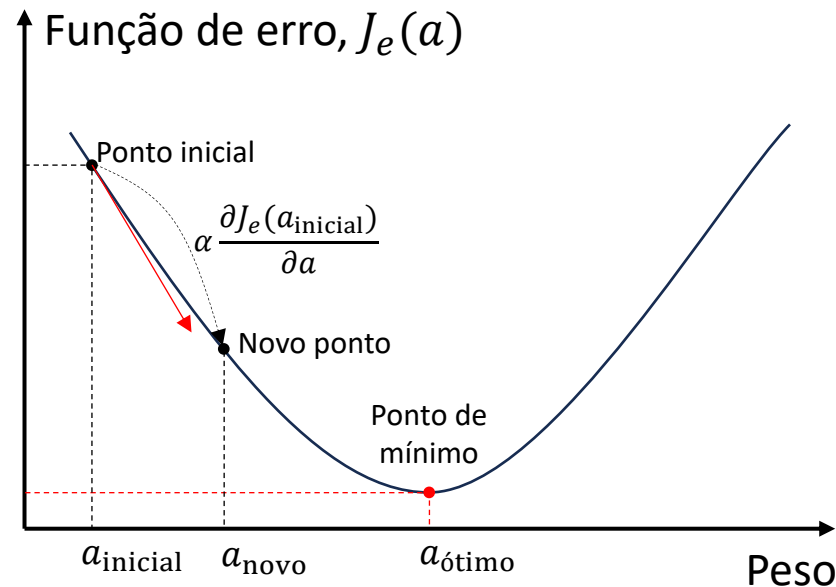


- O passo de aprendizagem, α , determina a **porcentagem da informação do vetor gradiente que é adicionada aos pesos**.
- A equação abaixo é chamada de **equação de atualização dos pesos**

$$a_{\text{novo}} = a_{\text{inicial}} - \alpha \frac{\partial J_e(a_{\text{inicial}})}{\partial a}$$

sentido oposto ao apontado pelo gradiente

Passo de aprendizagem

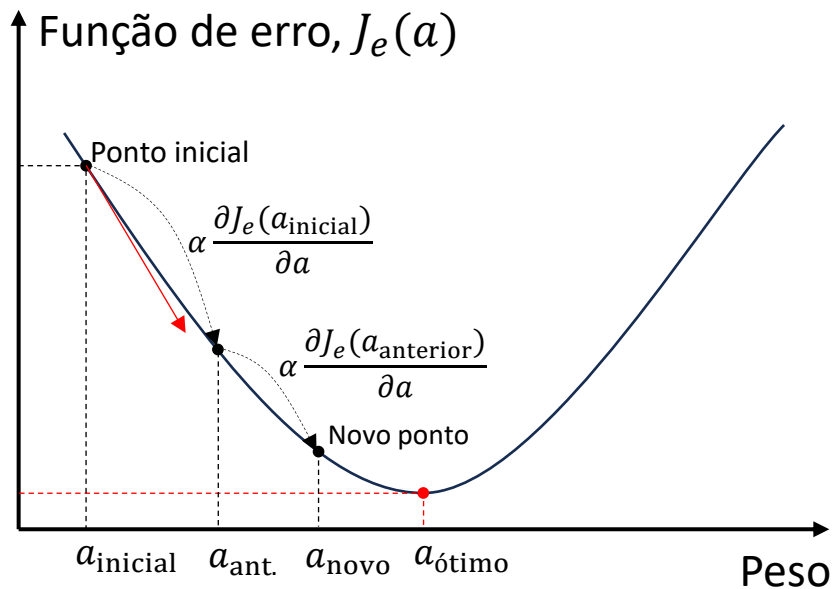


$$a_{novo} = a_{inicial} - \underbrace{\alpha \frac{\partial J_e(a_{inicial})}{\partial a}}_{\text{Termo de atualização}}$$

sentido apostado ao apontado pelo gradiente

- Se o peso atual está à esquerda do mínimo, o **termo de atualização** faz com que o **novo valor do peso seja maior** do que o anterior.
- Se o peso atual está à direita do mínimo, o **termo de atualização** faz com que o **novo valor do peso seja menor** do que o anterior.
- Portanto, dada a **direção do gradiente e um passo de aprendizagem**, agora podemos caminhar **iterativamente** em direção ao ponto de mínimo.

Otimização iterativa



- A cada **iteração** do GD calculamos o gradiente no ponto atual e atualizamos os pesos com uma porcentagem dele.
- **Repetimos** esse processo **até** que a **inclinação do hiperplano tangente ao ponto atual se torne igual a 0**, indicando que o ponto de mínimo foi atingido.
- **O que ocorre quando o gradiente é 0?**

$$a_{\text{novo}} = a_{\text{anterior}} - \alpha \frac{\partial J_e(a_{\text{anterior}})}{\partial a}$$

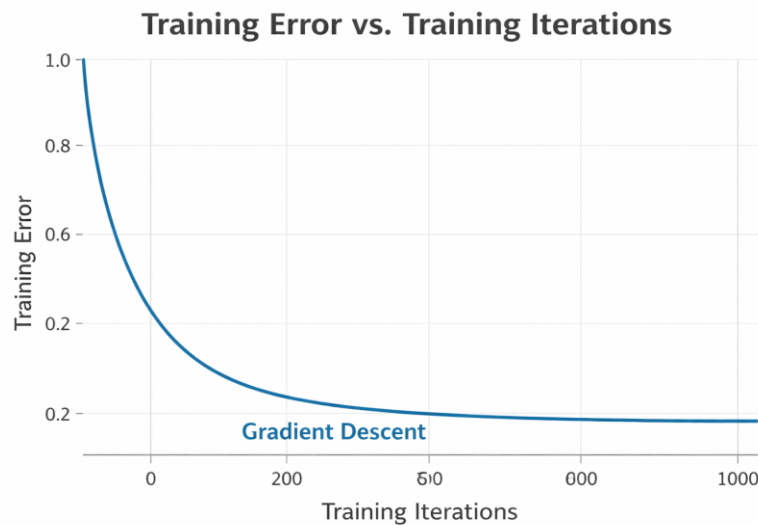
Equação de atualização dos pesos

Otimização iterativa

$$a_{\text{novo}} = a_{\text{anterior}} - \alpha \frac{\partial J_e(a_{\text{anterior}})}{\partial a}$$

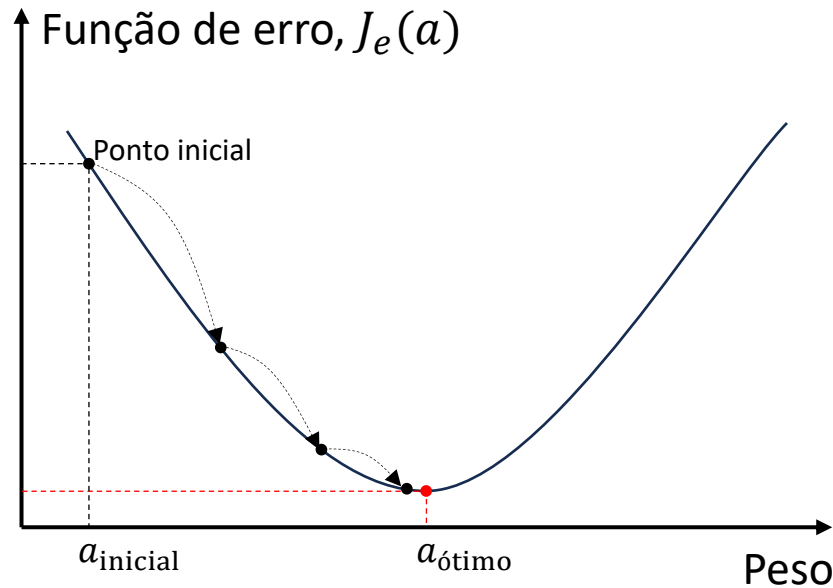


$$J_e(a) = \frac{1}{N} \|\mathbf{y} - X\mathbf{a}\|^2$$



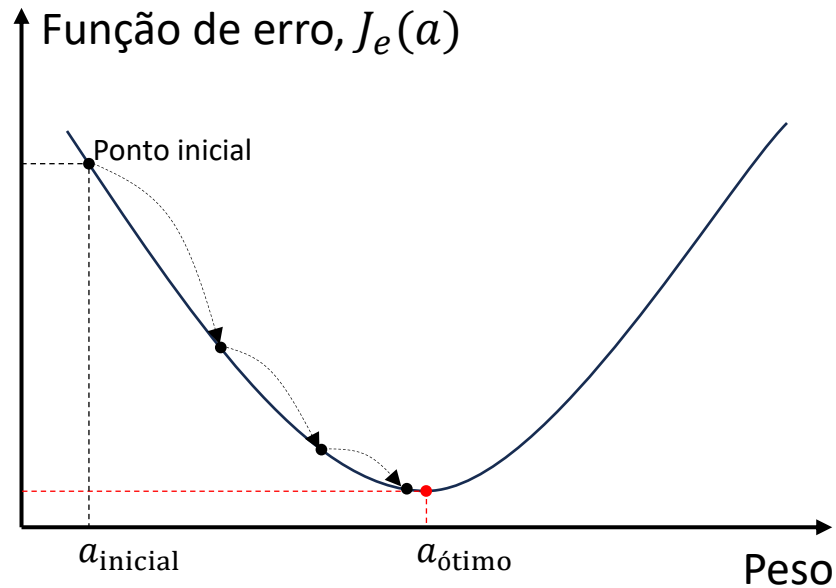
- Quando os elementos do vetor gradiente se tornam iguais a 0, os **pesos não são mais atualizados**, permanecendo **constantes**.
- Consequentemente, se os pesos são constantes, **o erro se torna constante**.
- Essa constância do erro (i.e., reta), é uma das formas usadas para encerrar o treinamento.
 - Se a **diferença do erro entre duas iterações consecutivas** cair abaixo de um valor bem pequeno, encerra-se o treinamento.

Tamanho do passo de aprendizagem



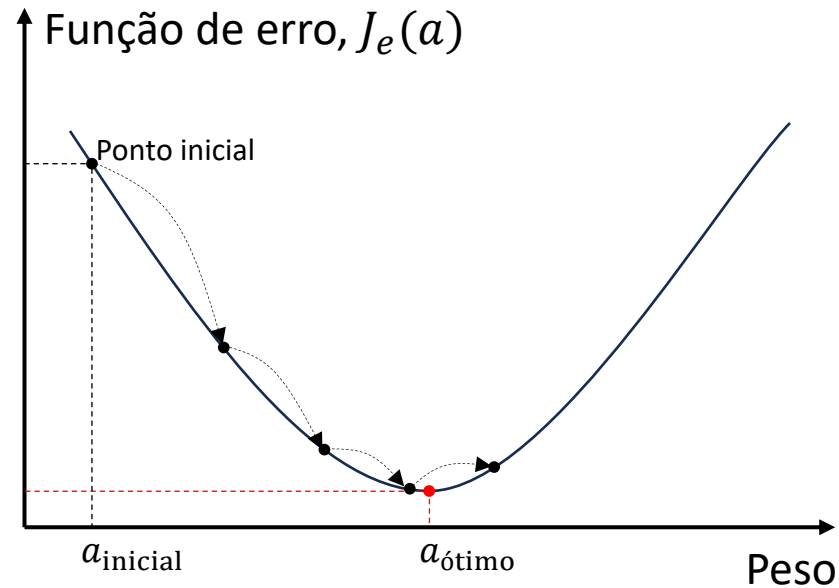
- O objetivo é que a cada nova iteração, o GD se mova para mais e mais perto do ponto de mínimo, $a_{\text{ótimo}}$.
- Porém, devemos tomar **cuidado**, com o **tamanho do passo de aprendizagem**.

Tamanho do passo de aprendizagem



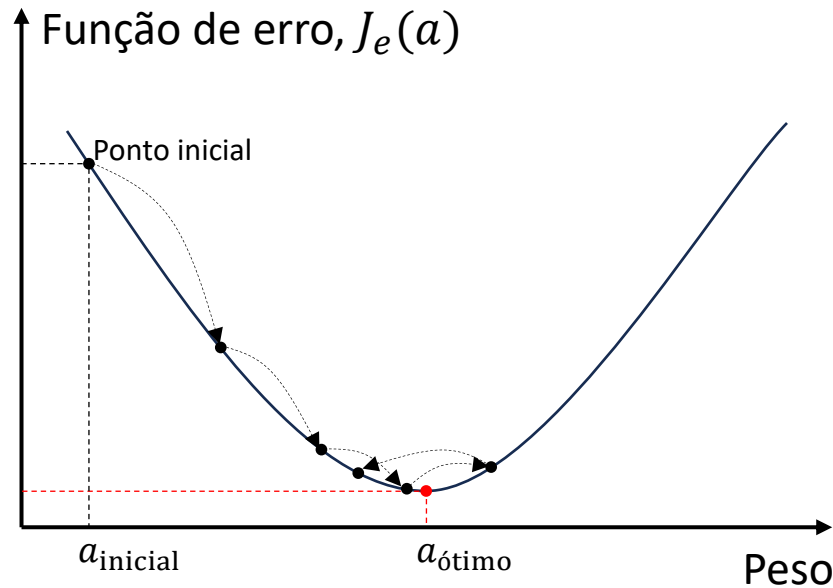
- O valor do passo de aprendizagem é um **hiperparâmetro** crucial para o GD.
 - **Hiperparâmetros** são parâmetros do modelo que **não são aprendidos durante o treinamento**, mas sim definidos antes do treinamento.
- O passo de aprendizagem **influencia a velocidade e a convergência** do treinamento.

Tamanho do passo de aprendizagem



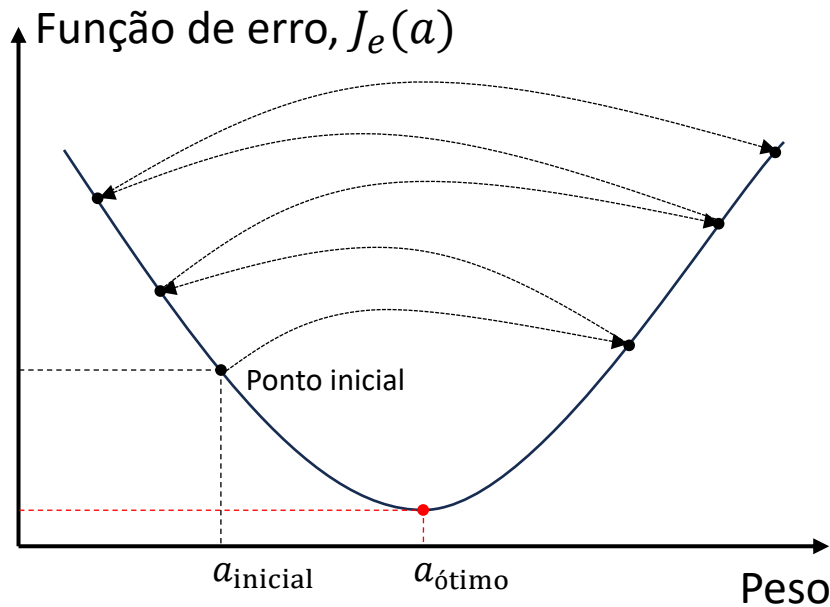
- Se o passo de aprendizagem for **muito grande**, podemos **ultrapassar** o ponto de mínimo.

Tamanho do passo de aprendizagem



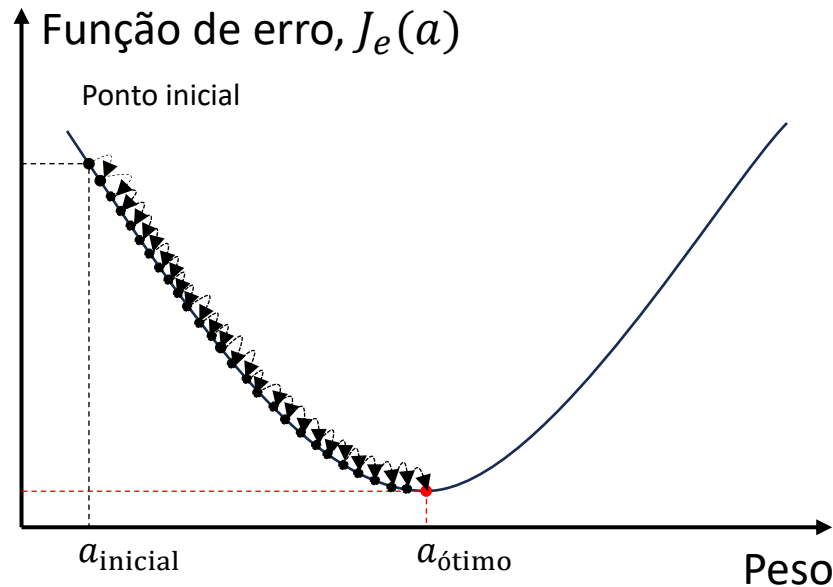
- Se o passo de aprendizagem for **muito grande**, o processo de otimização pode ficar **ziguezagueando** de um lado para o outro da base da função **sem nunca atingir o ponto de mínimo**.

Tamanho do passo de aprendizagem



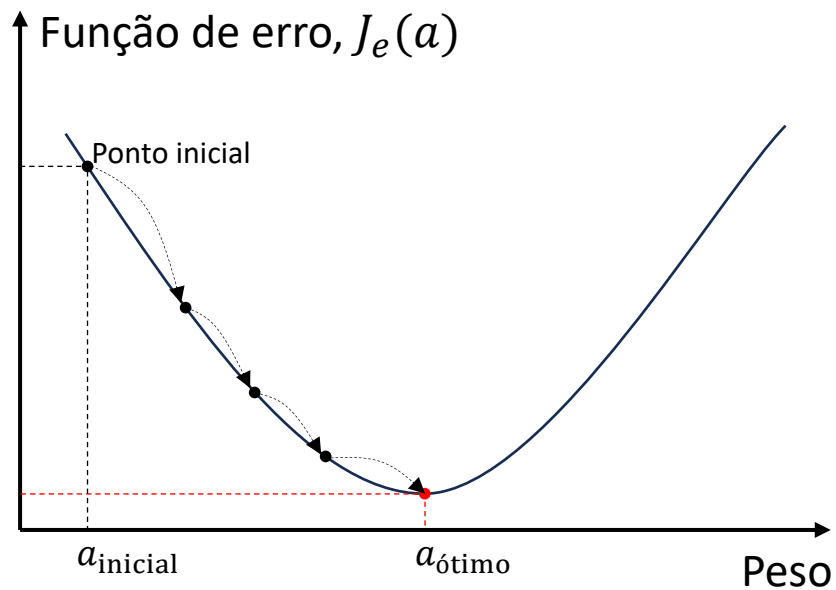
- Dependendo do quão grande for o valor do passo de aprendizagem, pode ocorrer até a **divergência** ao invés da convergência.
- Ou seja, ao invés de se aproximar do ponto de mínimo, o algoritmo **se distancia** dele a cada iteração.
- Se isso ocorrer, após algumas iterações, ocorre o **estouro da precisão numérica** das variáveis envolvidas na regra de atualização dos pesos.

Tamanho do passo de aprendizagem



- Outra questão, **menos problemática** que passos grandes, é a situação oposta.
- Passos **muito pequenos** fazem com que se **leve muito tempo**, i.e., iterações, para atingir o ponto de mínimo.
- Se cada iteração levar um tempo razoável para ser executada, o tempo necessário para se atingir o ponto de mínimo pode ser muito grande.
- Porém, **se esperarmos** tempo suficiente, a **convergência é garantida**.

Qual o tamanho de passo de aprendizagem usar?



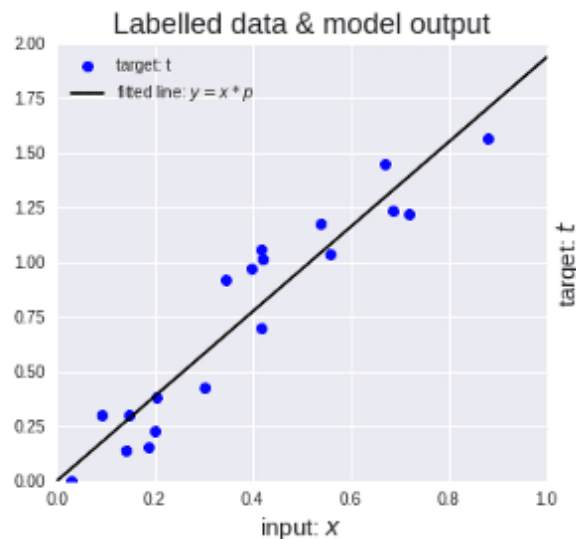
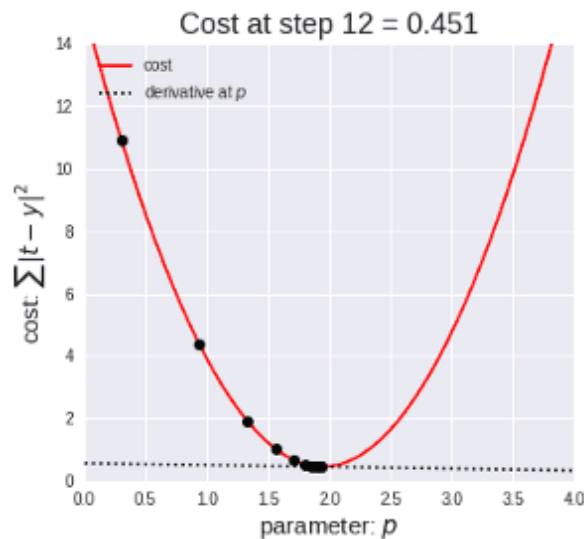
- É importante escolhermos um passo de aprendizagem que **acelere a convergência sem causar oscilações em torno do ponto de mínimo**.
- Em geral, um bom valor para o passo é encontrado por **tentativa e erro**.
- Uma forma mais avançada é ajustar o passo ao longo das iterações.
 - Usamos **valores maiores no início**, em pontos distantes do mínimo, e o **reduzimos gradualmente** ao longo das iterações.
 - Existem métodos que o ajustam automaticamente.

Pseudocódigo do gradiente descendente

$a \leftarrow$ inicializa em um ponto qualquer do espaço de pesos
loop até convergir ou atingir o número máximo de iterações do

$$a \leftarrow a - \alpha \frac{\partial J_e(a)}{\partial a} \text{ (eq. de atualização dos pesos)}$$

- A animação ao lado mostra o **gradiente descendente atualizando os pesos** de um **modelo**, no caso, uma **reta**, para que ele se **ajuste da melhor forma possível aos dados** de treinamento (pontos azuis).



Versões do gradiente descendente

- O GD pode ser implementado de **3 formas diferentes dependendo de como calculamos o vetor gradiente**.
- Para entendermos as 3 versões, vamos primeiro encontrar o vetor gradiente, $\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}$, e substituí-lo na equação de atualização dos pesos.
- Vamos considerar o EQM como função de erro e um hiperplano como função hipótese

$$\hat{y}(n) = \underbrace{a_0 + a_1 x_1(n) + \dots + a_K x_K(n)}_{\text{Equação do hiperplano}} = \sum_{i=0}^K a_i x_i(n) = \mathbf{x}(n)^T \mathbf{a},$$

onde $\mathbf{x}(n) = [1 \ x_1(n) \ \dots \ x_K(n)] \in \mathbb{R}^{K+1 \times 1}$ é um vetor com todos os atributos da n -ésima amostra de treinamento.

Versões do gradiente descendente

- O **vetor gradiente da função de erro em relação aos pesos** é dado por

$$\begin{aligned}\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} &= \frac{\partial}{\partial \mathbf{a}} \left[\frac{1}{N} \sum_{n=0}^{N-1} (y(n) - \hat{y}(n))^2 \right] \\ &= -\frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \mathbf{x}(n) = -\frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).\end{aligned}$$

- **OBS.:** Notem que o **gradiente** é a **média dos gradientes** de todos os exemplos de treinamento.
- Usamos um hiperplano para encontrar o gradiente, mas todos os resultados e conclusões podem ser diretamente estendidos a polinômios.

Versões do gradiente descendente

- Substituindo o **vetor gradiente** na **equação de atualização dos pesos**, temos

$$\mathbf{a} = \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \mathbf{a} + \alpha \frac{2}{N} \sum_{n=0}^{M-1} [y(n) - \hat{y}(n)] \mathbf{x}(n).$$

- Podemos ter **3 versões diferentes, dependendo da quantidade de amostras, M** , considerados na média acima:
 - Gradiente descendente em batelada (GDB).
 - Gradiente descendente estocástico (GDE).
 - Gradiente descendente em mini-lotes (GDML).

Gradiente descendente em batelada

$$\mathbf{a} = \mathbf{a} + \alpha \frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \mathbf{x}(n).$$

- Calcula o gradiente usando **todas as amostras** de treinamento ($M = N$), resultando em uma **descida suave e direta** em direção ao mínimo.
- **Convergência** para o **mínimo global** é **garantida** quando a função de erro é **convexa e o passo não é grande demais**.
- Pode ser **computacionalmente complexo** dependendo do tamanho do modelo e do conjunto de dados.
 - Por processar todos os exemplos, é lento em conjuntos muito grandes e consome muita memória.
- Das 3, é a versão que **obtem os melhores resultados**.

Gradiente descendente estocástico

$$\mathbf{a} = \mathbf{a} + \alpha 2[y(n_{\text{random}}) - \hat{y}(n_{\text{random}})]\mathbf{x}(n_{\text{random}}).$$

- Utiliza **apenas uma amostra** do conjunto de treinamento (i.e., $M = 1$) para calcular uma **estimativa estocástica do gradiente**.
 - Versão estocástica, pois a cada iteração toma-se uma amostra **aleatória** do conjunto para calcular a estimativa do gradiente.
- Quando os **dados de treinamento são ruidosos**, a **estimativa** do gradiente **é ruidosa**, fazendo com que a **convergência não ocorra** ou **não seja garantida**.
- Atualiza os pesos a cada exemplo processado, gerando uma trajetória oscilatória que flutua ao redor do mínimo devido à estimativa ruidosa do gradiente.
- Entretanto, é mais rápido e usa menos CPU e memória do que o GDB.

Gradiente descendente em mini-lotes

$$\mathbf{a} = \mathbf{a} + \alpha \frac{2}{MB} \sum_{n=0}^{MB-1} [y(n) - \hat{y}(n)] \mathbf{x}(n).$$

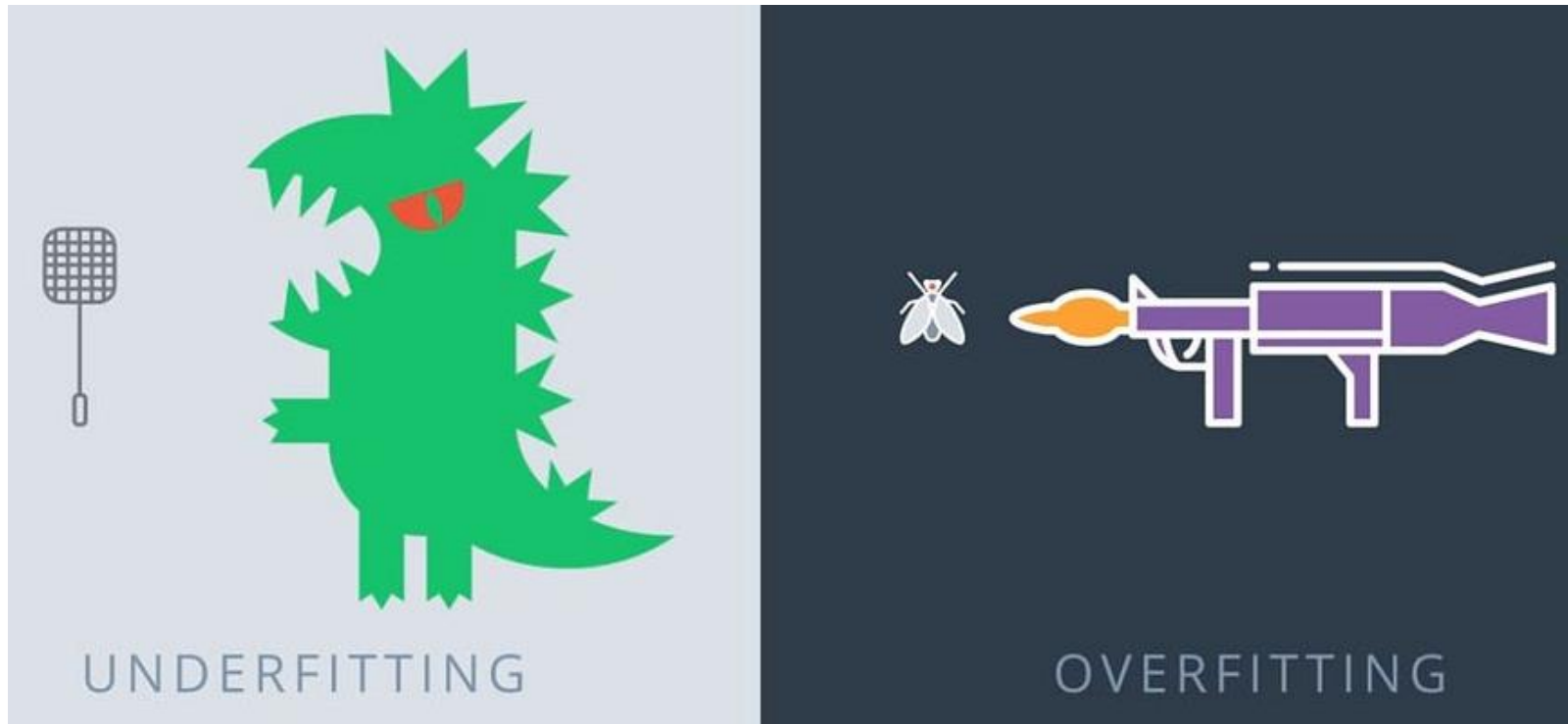
- Utiliza um **subconjunto de amostras**, MB , do conjunto de treinamento ($M = MB$) para o cálculo do gradiente.
- Por, em geral, usar um **subconjunto** de exemplos, $1 < MB < N$, é mais rápido que o GDB e mais preciso e estável do que o GDE.
- Porém, por MB ser variável, essa versão é vista como uma **generalização** das duas versões anteriores, pois MB pode ser feito igual a 1 ou N .
- Portanto, por poder ter sua complexidade configurada, é a **versão mais usada no treinamento de redes neurais**.

Já sabemos como encontrar os pesos de um dado polinômio, mas como descobrimos a complexidade ideal, i.e., o grau deste polinômio?

Underfitting e overfitting

- Antes de falarmos sobre a validação cruzada, que é a forma mais usada para encontrar o melhor modelo, precisamos discutir sobre alguns comportamentos que vamos encontrar durante essa busca.
- Dependendo da complexidade (ou graus de liberdade) do modelo, no caso da regressão, do grau do polinômio, podemos nos deparar com dois comportamentos indesejados:
 - *Underfitting* (subajuste)
 - *Overfitting* (sobreajuste)

Overfitting e underfitting



Underfitting: o modelo é simples demais para a complexidade do problema.

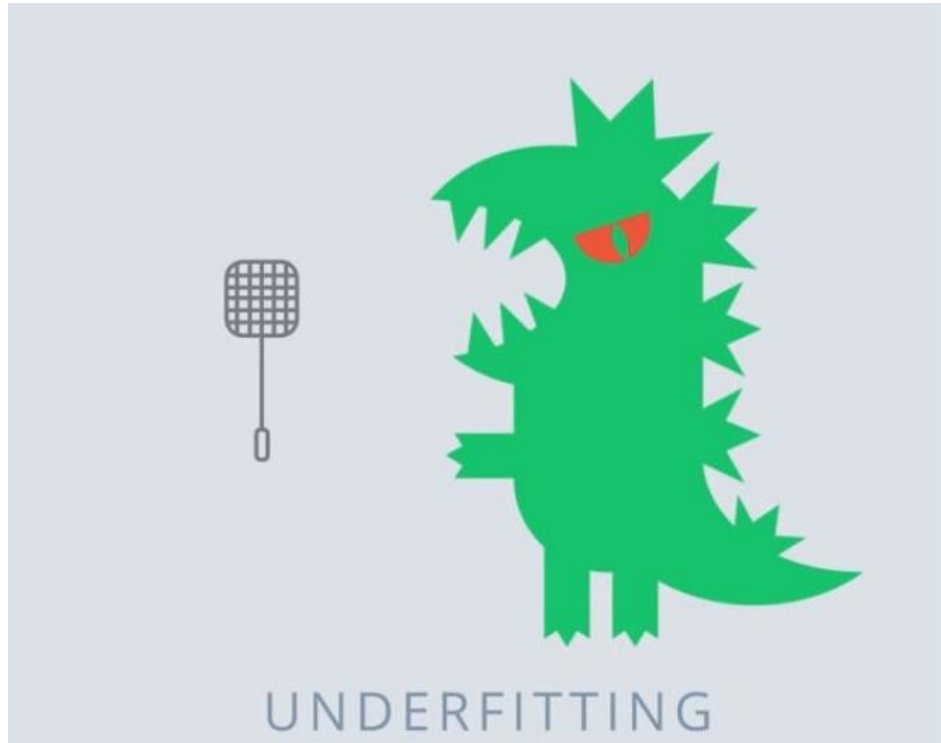
Overfitting: o modelo é complexo demais.

Overfitting



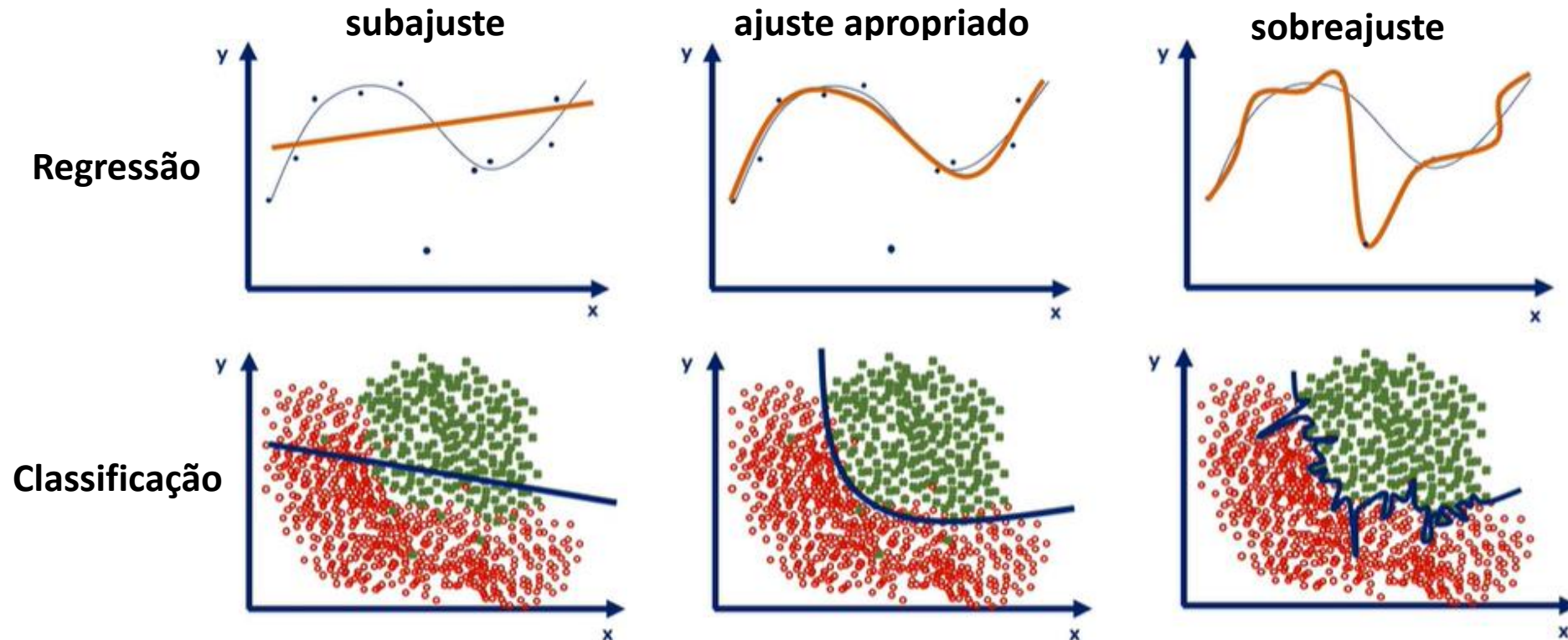
- O modelo é tão flexível (i.e., complexo) que “**memoriza**” o conjunto de treinamento.
- Ele se ajusta ao ruído em vez do padrão geral por trás dos dados.
- **Alta variância:** o modelo é extremamente sensível ao conjunto de treino.
 - Pequenas mudanças nas amostras de treinamento geram grandes mudanças no modelo, no caso da regressão, da função estimada.
- O modelo não generaliza bem.

Underfitting



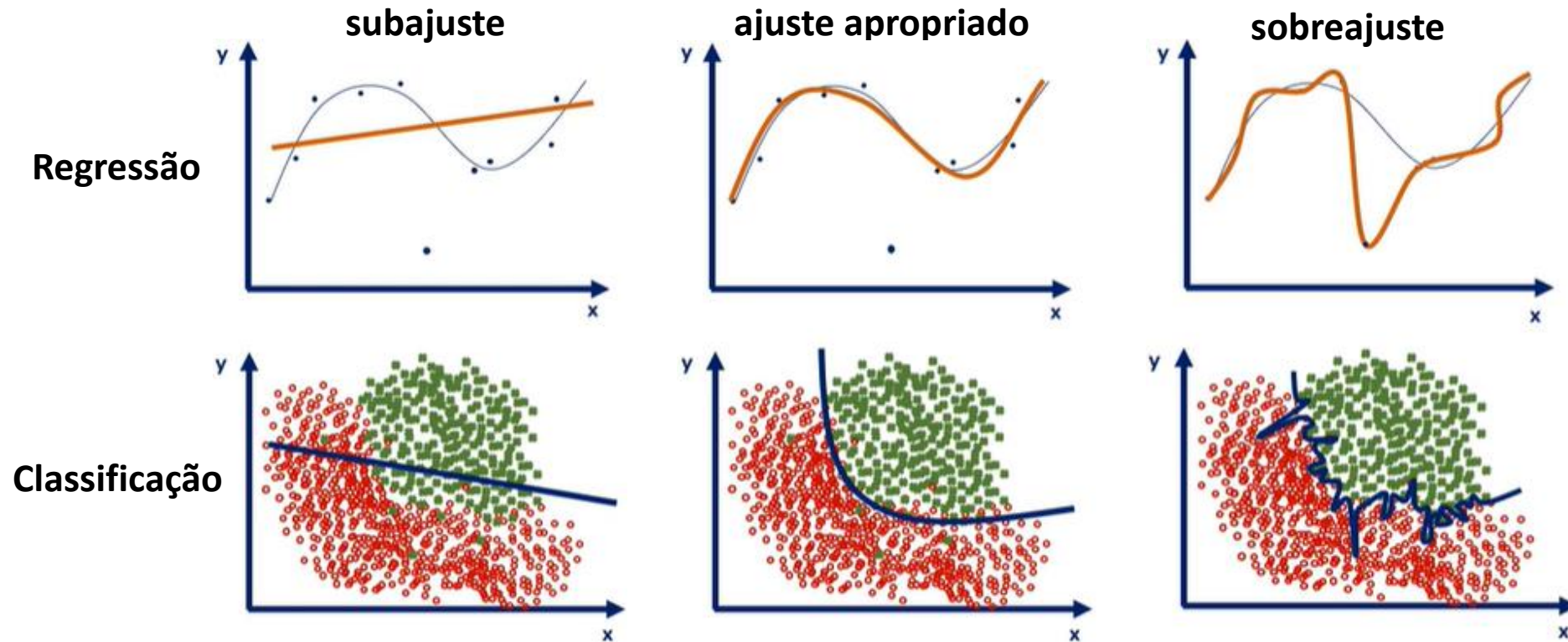
- O modelo é **muito simples** (i.e., tem pouca flexibilidade) para aprender/capturar o padrão geral por trás dos dados de treinamento.
- **Alto viés:** faz suposições muito fortes sobre a forma da função (e.g., assumir linearidade quando a relação é curva).
- O modelo não generaliza bem.

Underfitting



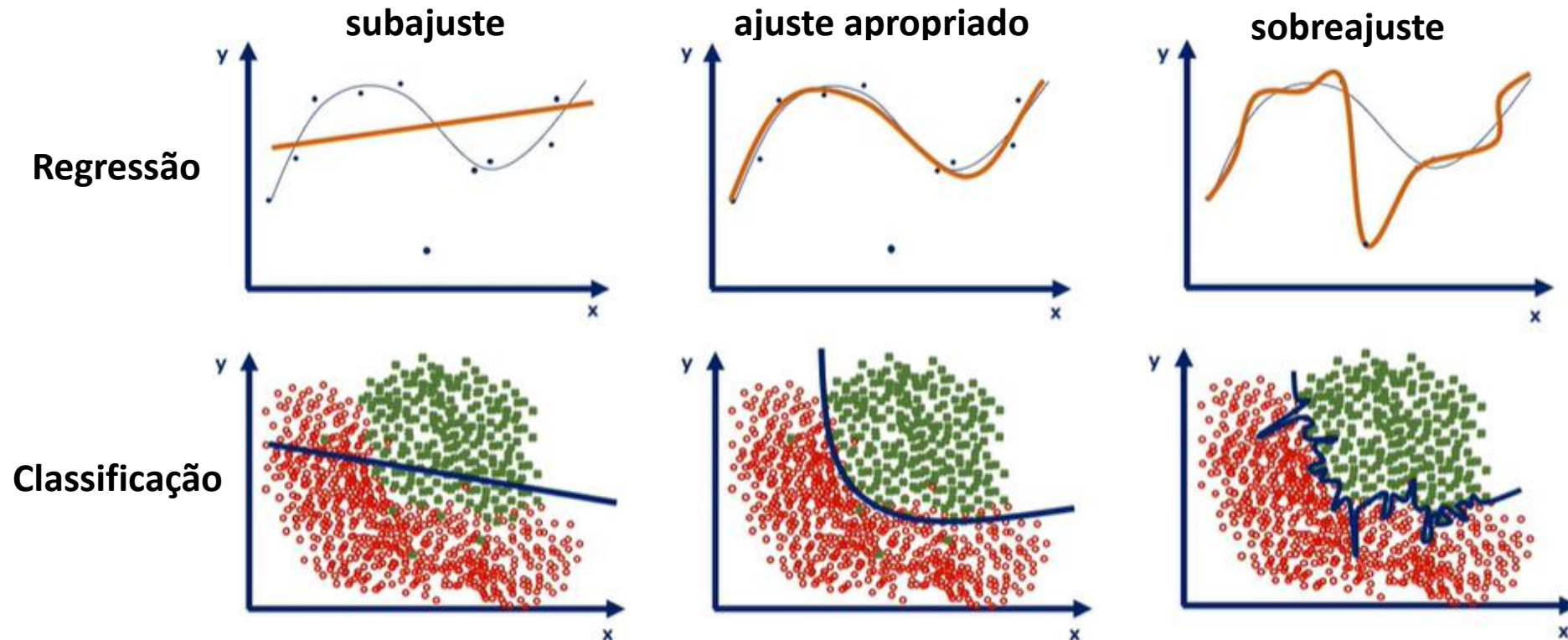
- O modelo **não é capaz de capturar os padrões** presentes nos dados de treinamento.
- O modelo apresenta **erros altos nos conjunto de treinamento e validação**.
- Esse padrão indica alto viés.

Underfitting



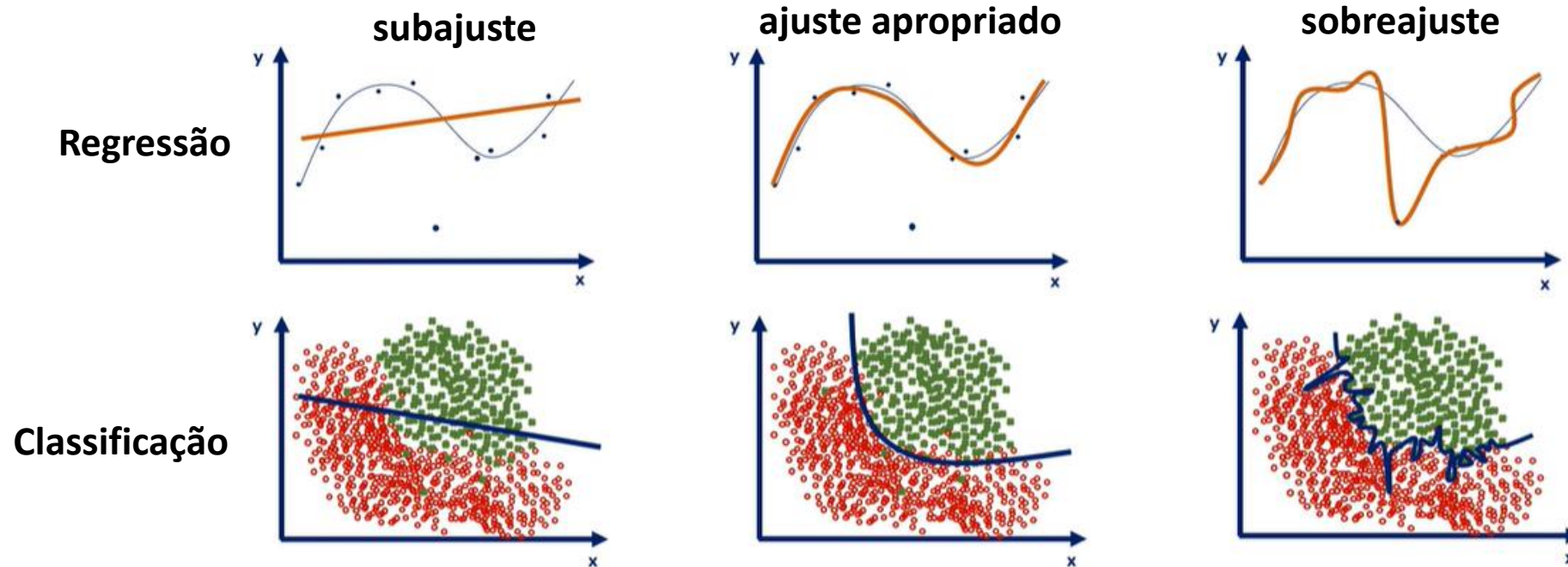
- Algumas causas: modelo **sem complexidade**, **poucas épocas de treinamento** e **falta de dados** (modelo falha em aprender as características relevantes).

Underfitting



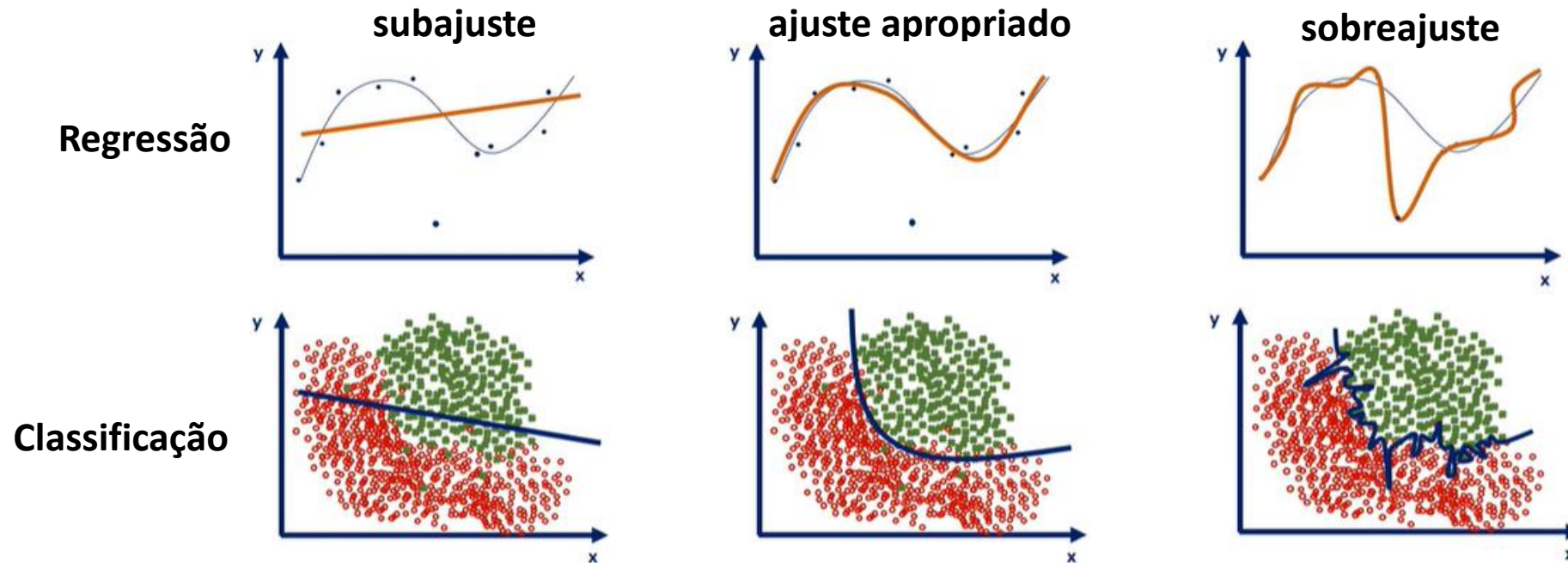
- Como mitigar: **aumentar a complexidade** do modelo (e.g., grau do polinômio), **ajustar os hiperparâmetros** (e.g., passo de aprendizagem), **treinar por mais épocas** e **aumentar o conjunto de treinamento**, se possível.

Overfitting



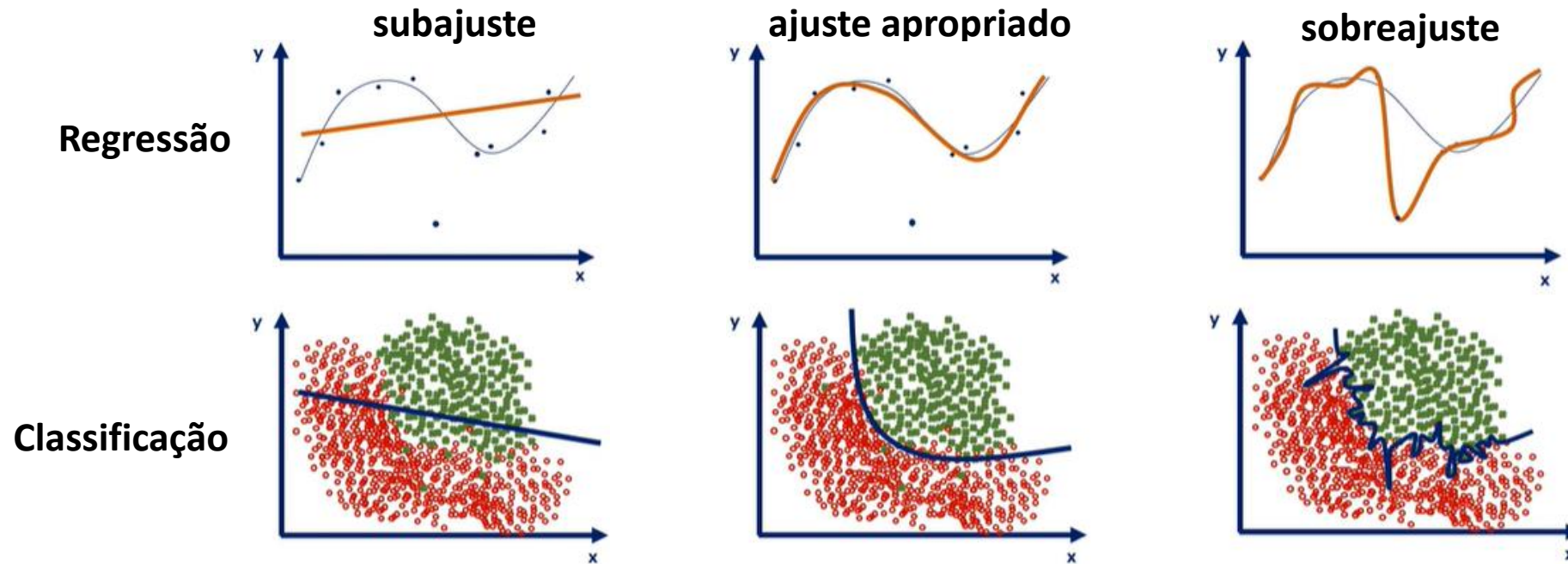
- O modelo se ajusta excessivamente aos dados de treinamento e perde a capacidade de generalizar para dados inéditos.
- O modelo apresenta **erro muito baixo**, tendendo a zero, **no conjunto de treinamento e muito alto no conjunto de validação**.
- Esse padrão indica alta variância.

Overfitting



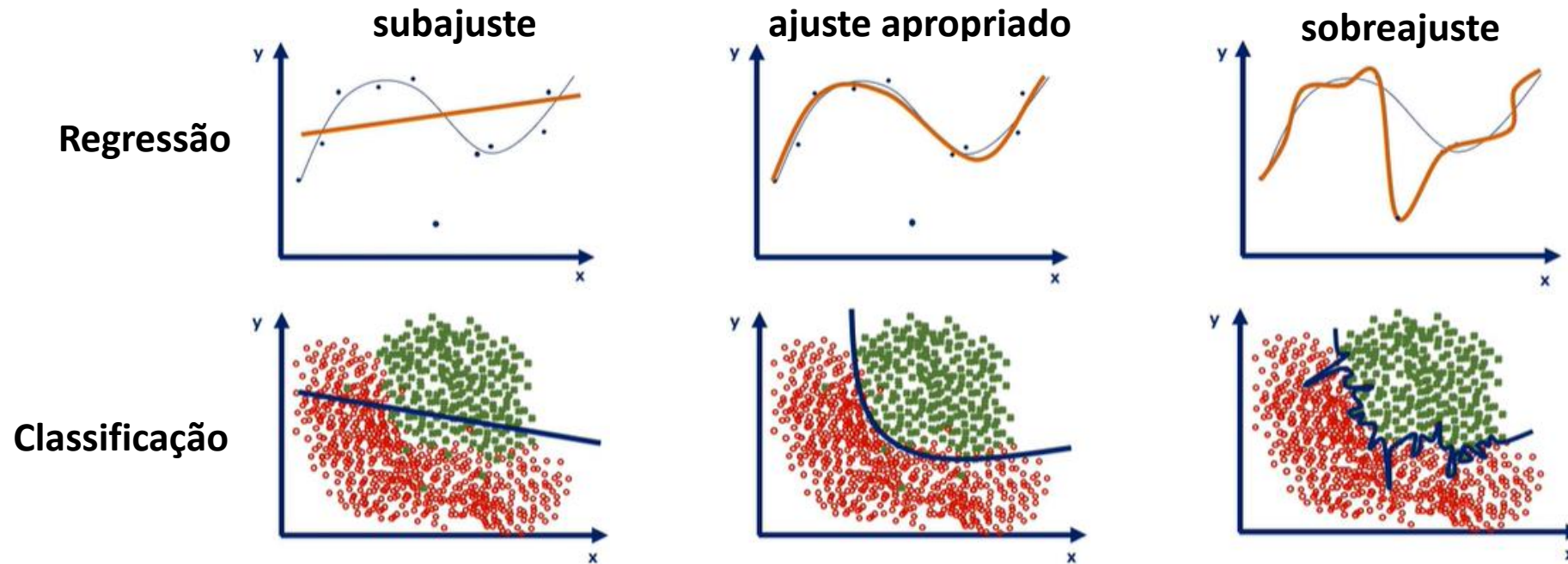
- Algumas causas: **modelo muito complexo** em relação à quantidade de dados, **treinamento excessivo** (leva à memorização dos dados de treinamento), **falta de dados** (modelo tem poucos exemplos para aprender os padrões gerais).

Overfitting



- Como mitigar: **aumentar os dados de treinamento**, se possível, **reduzir a complexidade do modelo** (e.g., reduzir grau do polinômio), **aplicar técnicas de regularização** (e.g., penalizações L1 ou L2, **early-stopping**, o qual encerra o treinamento quando o modelo começa sobreajustar).

Overfitting e underfitting



- Encontrar o **equilíbrio entre a complexidade** (ou flexibilidade) do modelo e sua **capacidade de generalização** é essencial para um bom desempenho.
- Ou seja, devemos encontrar um **equilíbrio entre** um **modelo muito simples** (subajuste) e um **modelo muito complexo** (sobreajuste).

Validação cruzada

- A validação cruzada é uma técnica utilizada para **avaliar quantitativamente o desempenho** de um **modelo** e **garantir que ele generalize bem para dados inéditos**, evitando assim problemas de subajuste ou sobreajuste.
- O processo de validação cruzada envolve **dividir o conjunto total de dados em subconjuntos** e realizar **algumas rodadas de treinamento e validação** do modelo com **combinações desses subconjuntos**.
- A validação cruzada é uma ferramenta importante para **comparar e selecionar modelos** e para **ajustar hiperparâmetros** como, por exemplo, o **passo de aprendizagem**, o **grau do polinômio** da função hipótese, etc.

Validação cruzada

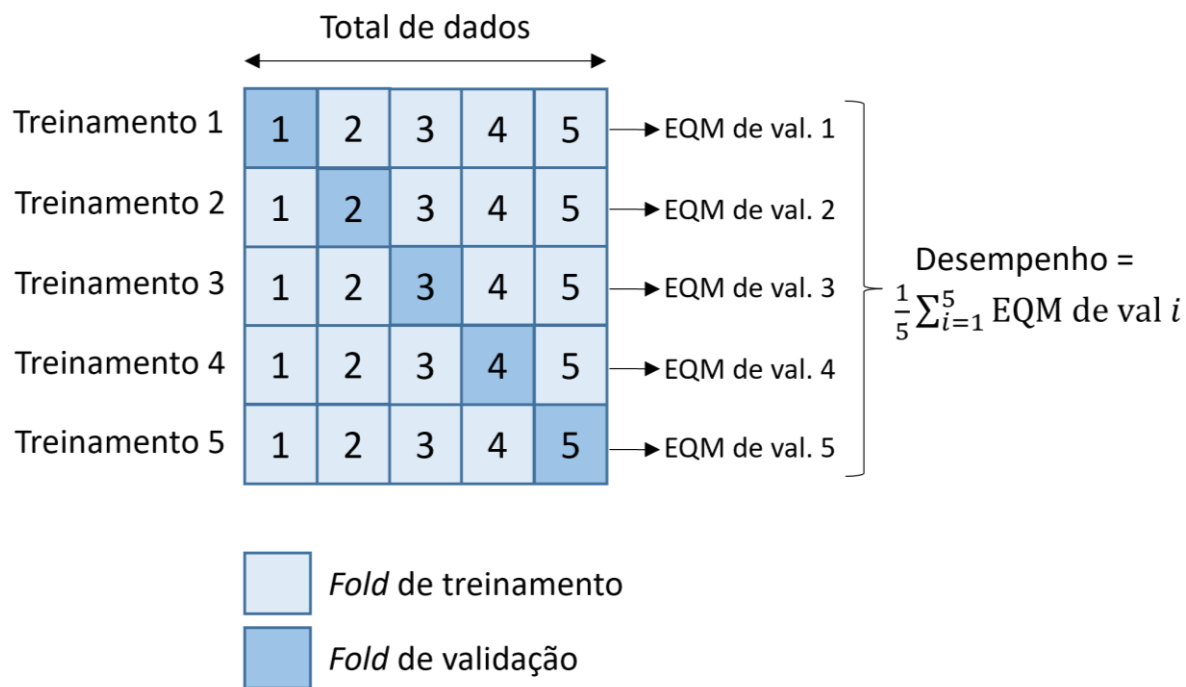
- O **objetivo** da **validação cruzada** é encontrar um **ponto de equilíbrio** entre a **flexibilidade** e a **capacidade de generalização** do modelo (e.g., polinômio).
- Um **modelo equilibrado** é
 - Flexível o **suficiente** para se ajustar ao **comportamento geral** dos dados.
 - Capaz de prever saídas próximas às esperadas para exemplos não vistos durante seu treinamento.
- A **flexibilidade** de um modelo é **estimada** através do **erro de treinamento** e a **capacidade de generalização** é **estimada** através do **erro de validação** ou de **teste**.
 - Erro de treinamento é calculado com o conjunto usado para treinar o modelo.
 - Erro de validação ou teste é calculado com dados inéditos.

Validação cruzada

- No caso onde queremos usar a **validação cruzada** para **encontrar o grau ideal da função hipótese polinomial**, o **comportamento destes dois erros** vai nos ajudar a verificar **quais graus fazem o modelo se ajustar demais ou insuficientemente** aos dados de treinamento.
- A estratégia de validação cruzada mais utilizada é conhecida como *k-fold*.
- Existem outras, como *holdout* e *leave-p-out*, mas focaremos no *k-fold*.

k -fold

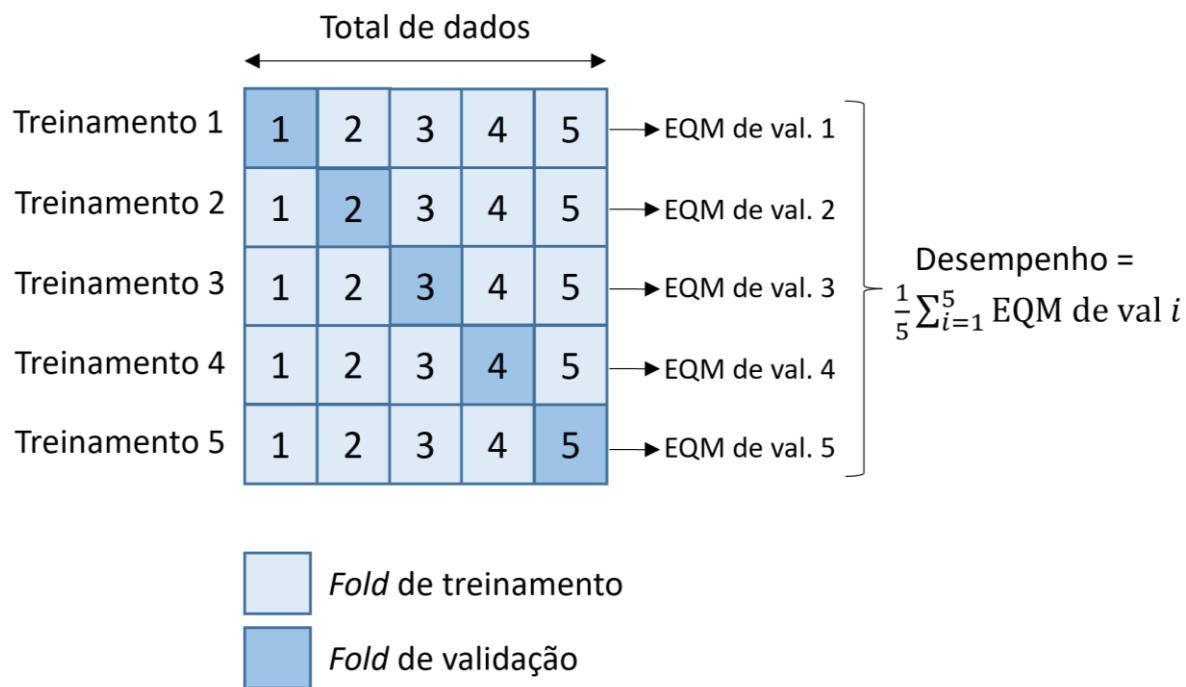
$k = 5$



- A estratégia consiste em embaralhar (opcional) e **dividir o conjunto total de dados em k partes** (ou *folds*) **iguais**.
- O **modelo é treinado k vezes**, cada vez usando **$k-1$** partes como conjunto de treinamento e a parte restante como conjunto de validação.
- O **EQM com cada conjunto de validação** é calculado **ao final de cada treinamento**.

k -fold

$k = 5$

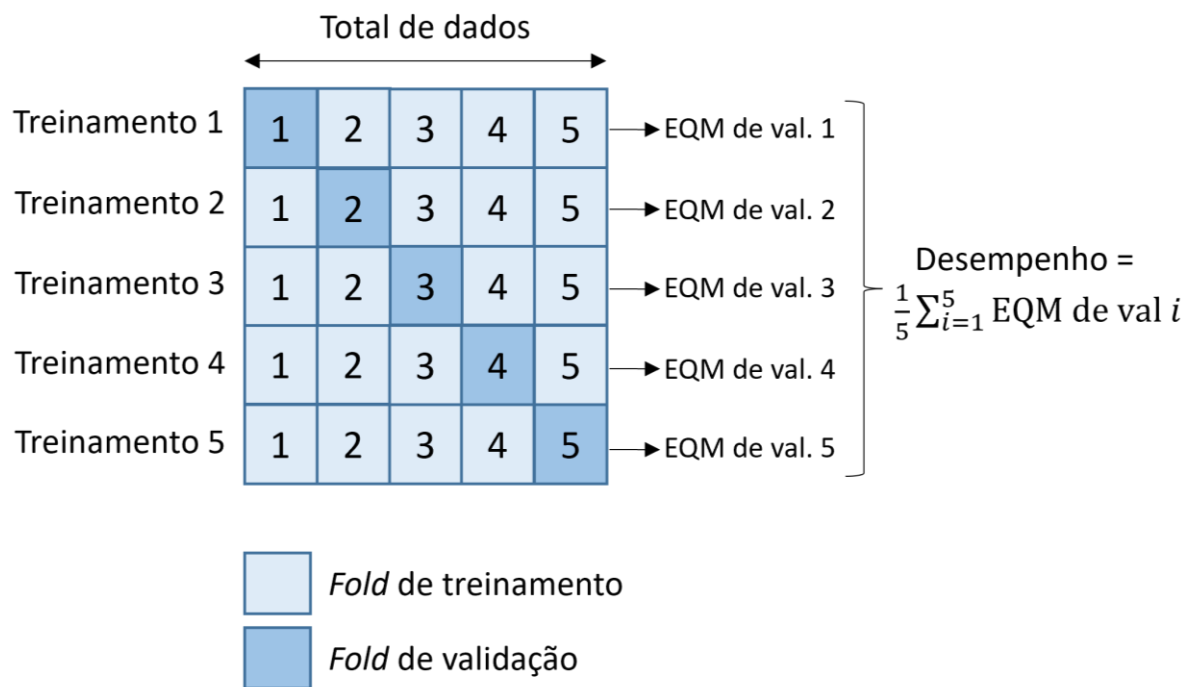


- Ao final dos k treinamentos, calcula-se a **média** e o **desvio padrão** dos k EQMs de validação para fornecer uma **avaliação geral do desempenho do modelo**:

- A média fornece uma estimativa da capacidade de generalização do modelo em dados inéditos.
- O desvio padrão indica a estabilidade do modelo diante de diferentes subconjuntos de treinamento.

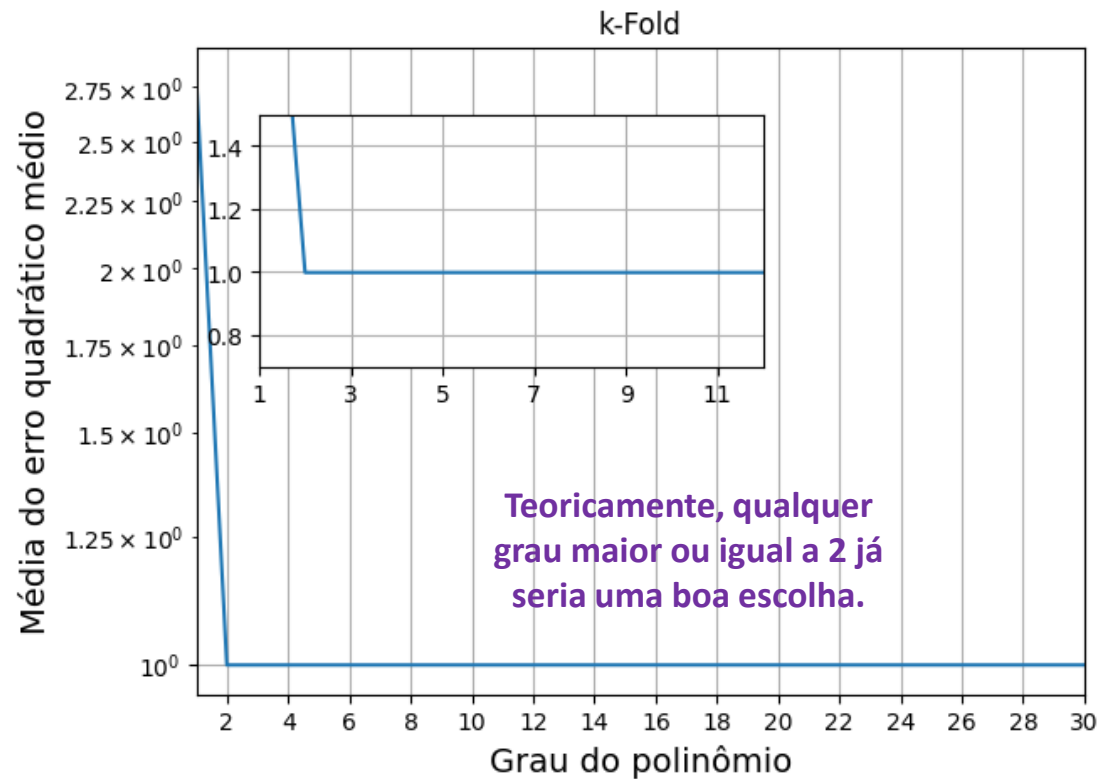
k -fold

$k = 5$



- Em geral, utiliza-se $k = 5$ ou 10.
- Porém, k deve ser escolhido de forma que os folds sejam **representativos do padrão presente nos dados**.

Qual grau escolher quando vários são possíveis?

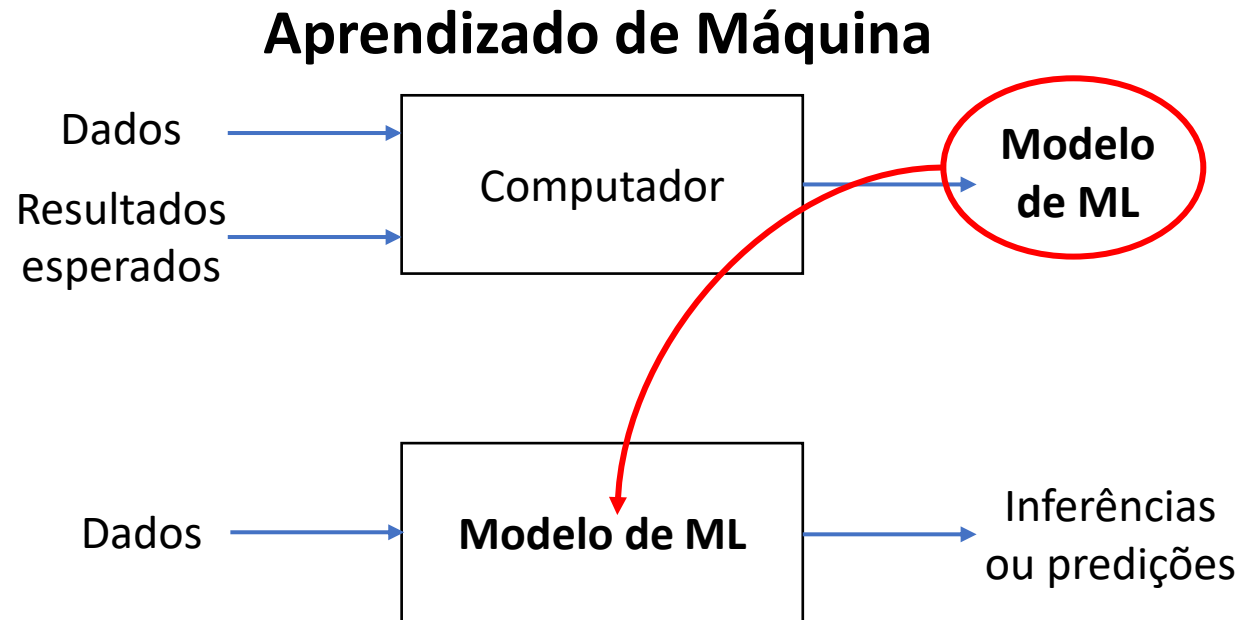


- Qual grau devemos escolher quando a média dos erros são mínimos e praticamente constantes para vários graus de polinômio?
- Essa situação ocorre quando o número de amostras é muito maior do que a flexibilidade (i.e., grau) do modelo.

Qual grau escolher quando vários são possíveis?

- A resposta é aplicar o **princípio da navalha de Occam**.
- A **navalha de Occam** é um princípio lógico que sugere que, **entre várias explicações igualmente plausíveis** para um conjunto de observações, a **mais simples deve ser preferida**.
 - Ou seja, deve-se **preferir explicações mais simples às mais complexas**.
- Portanto, usando a **navalha de Occam** escolhemos a **função hipótese polinomial com menor grau** (i.e., menos complexa), **mas que se ajusta bem ao comportamento geral dos dados**.
 - Ou seja, escolhemos o modelo mais simples em termos de quantidade de cálculos, mas que possua uma boa capacidade de generalização.

Inferência



- Após termos treinado e encontrado o modelo, o usamos para fazer **inferências**.
- Ou seja, o usamos para fazer **previsões** (i.e., valores de saídas) **para entradas inéditas**.

Generalização

- Porém, não basta que o algoritmo de ML treine um modelo que faça um bom mapeamento **apenas para os dados do conjunto de treinamento**.
- O **algoritmo** de ML deve **treinar** um **modelo** que **aprenda** uma **solução geral**, ou seja, um modelo que **faça previsões precisas para entradas não vistas durante o treinamento**.
- Essa extrapolação do conhecimento adquirido durante o treinamento é chamado de **generalização**.



Exemplo

- [Exemplo: regressão.ipynb](#)



Avisos

- NP1: 13/04/2026
- 10 questões de múltipla escolha.
- Conteúdo: até o que for apresentado na aula do dia 06/04/2026.

Lista de exercícios

[Lista #5 - Regressão](#)

Perguntas?

Obrigado!

Anexo:

Cálculo do vetor gradiente

Cálculo do vetor gradiente

Considerando o hiperplano como a função hipótese

$$\hat{y}(n) = \mathbf{x}(n)^T \mathbf{a}.$$

O vetor gradiente é calculado como

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \left[\frac{\partial J_e(\mathbf{a})}{\partial a_0} \quad \dots \quad \frac{\partial J_e(\mathbf{a})}{\partial a_K} \right]^T.$$

Assim, o vetor gradiente da função de erro em relação aos pesos é dado por

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \frac{\partial}{\partial \mathbf{a}} \left[\frac{1}{N} \sum_{n=0}^{N-1} (y(n) - \hat{y}(n))^2 \right].$$

Cálculo do vetor gradiente

Como a operação de derivada é distributiva, podemos reescrever a equação acima como

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \frac{1}{N} \sum_{n=0}^{N-1} \frac{\partial (y(n) - \hat{y}(n))^2}{\partial \mathbf{a}}.$$

Substituindo a função hipótese na equação acima, temos

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \frac{1}{N} \sum_{n=0}^{N-1} \frac{\partial (y(n) - \mathbf{a}^T \mathbf{x}(n))^2}{\partial \mathbf{a}}.$$

Cálculo do vetor gradiente

Aplicando a regra da cadeia, reescrevemos a equação anterior como

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{2}{N} \sum_{n=0}^{N-1} (y(n) - \mathbf{a}^T \mathbf{x}(n)) \frac{\partial \mathbf{a}^T \mathbf{x}(n)}{\partial \mathbf{a}}.$$

Sabendo que a derivada de $\frac{\partial \mathbf{a}^T \mathbf{x}(n)}{\partial \mathbf{a}}$ é igual a $\mathbf{x}(n)$, reescrevemos a equação anterior como

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \mathbf{x}(n).$$

Cálculo do vetor gradiente

Fazendo $y(n) - \hat{y}(n) = d(n)$

$$\begin{aligned}\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} &= -\frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \mathbf{x}(n) \\ &= -\frac{2}{N} \left\{ d(0) \begin{bmatrix} x_0(0) \\ \vdots \\ x_K(0) \end{bmatrix} + d(1) \begin{bmatrix} x_0(1) \\ \vdots \\ x_K(1) \end{bmatrix} + \cdots + d(N-1) \begin{bmatrix} x_0(N-1) \\ \vdots \\ x_K(N-1) \end{bmatrix} \right\} \\ &= -\frac{2}{N} \begin{bmatrix} d(0)x_0(0) + d(1)x_0(1) + \cdots + d(N-1)x_0(N-1) \\ \vdots \\ d(0)x_K(0) + d(1)x_K(1) + \cdots + d(N-1)x_K(N-1) \end{bmatrix}.\end{aligned}$$

Notem que a equação acima é um **vetor coluna** com dimensão $K + 1 \times 1$.

Cálculo do vetor gradiente

Podemos reescrever a equação (i.e., vetor) anterior como uma multiplicação matricial

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{2}{N} \begin{bmatrix} x_0(0) & x_0(1) & \cdots & x_0(N-1) \\ \vdots & \vdots & & \vdots \\ x_K(0) & x_K(1) & \cdots & x_K(N-1) \end{bmatrix} \begin{bmatrix} d(0) \\ d(1) \\ \vdots \\ d(N-1) \end{bmatrix}$$

Percebam que temos a multiplicação de uma matriz com dimensão $K + 1 \times N$ por um vetor coluna de dimensão $N \times 1$.

A matriz contém em cada linha todos os valores de $n = 0$ a $n = N - 1$ de um **único** atributo.

O vetor contém em cada linha a diferença $d(n) = y(n) - \hat{y}(n)$ para $n = 0$ até $n = N - 1$.

Cálculo do vetor gradiente

Se definirmos uma matriz que contém todos os N exemplos de todos os $K + 1$ atributos e que tem dimensão $N \times K + 1$

$$\mathbf{X} = \begin{bmatrix} x_0(0) & \cdots & x_K(0) \\ x_0(1) & \cdots & x_K(1) \\ \vdots & & \vdots \\ x_0(N-1) & \cdots & x_K(N-1) \end{bmatrix},$$

e dois vetores coluna com dimensões $N \times 1$ contendo todos os valores esperados (i.e., rótulos) e todos os valores preditos

$$\mathbf{y} = \begin{bmatrix} y(0) \\ \vdots \\ y(N-1) \end{bmatrix} \quad \text{e} \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}(0) \\ \vdots \\ \hat{y}(N-1) \end{bmatrix}$$

Cálculo do vetor gradiente

Usando a matriz e os vetores definidos no slide anterior, podemos reescrever o vetor gradiente como

$$\begin{aligned} & \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} \\ &= -\frac{2}{N} \begin{bmatrix} x_0(0) & x_0(1) & \cdots & x_0(N-1) \\ \vdots & \vdots & & \vdots \\ x_K(0) & x_K(1) & \cdots & x_K(N-1) \end{bmatrix} \left(\begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(N-1) \end{bmatrix} - \begin{bmatrix} \hat{y}(0) \\ \hat{y}(1) \\ \vdots \\ \hat{y}(N-1) \end{bmatrix} \right) \\ &= -\frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}) \end{aligned}$$

O resultado da multiplicação matricial acima continua resultando em um **vetor coluna** com dimensão $K + 1 \times 1$, ou seja, $(K + 1 \times N) \times (N \times 1) = K + 1 \times 1$.

Equação de atualização dos pesos

Utilizando o resultado anterior, podemos reescrever a equação de atualização dos pesos como

$$\begin{aligned}\mathbf{a} &= \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} \\ &= \mathbf{a} + \alpha \frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).\end{aligned}$$

A soma acima deve resultar em um vetor coluna com dimensão $K + 1 \times 1$, pois esta é a dimensão dos dois vetores sendo somados.

Lembrem-se que $K + 1 \times 1$ é a dimensão do vetor $\mathbf{a}$, o qual contém todos os pesos do modelo e que $\frac{2}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}})$ tem dimensão $K + 1 \times 1$ também.

Equação de atualização dos pesos

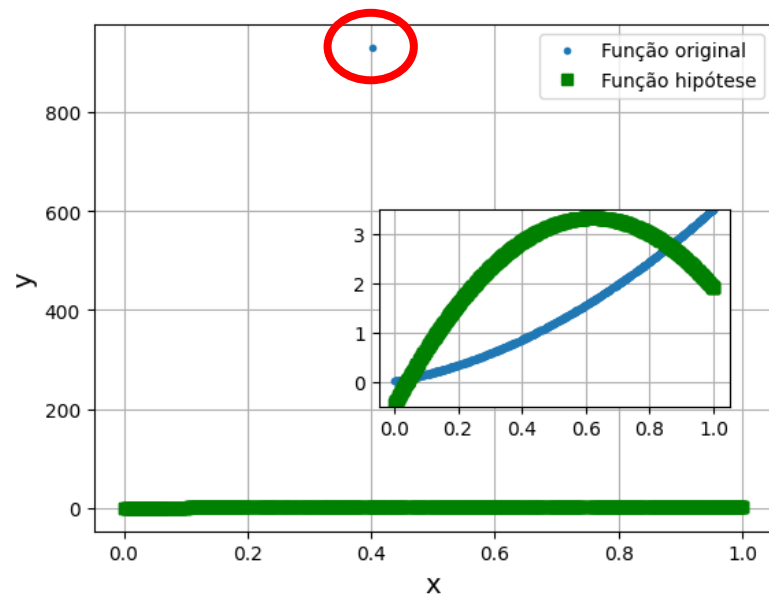
Podemos reescrever a equação de atualização dos pesos como

$$\begin{aligned}
 \mathbf{a} &= \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = \begin{bmatrix} a_0 \\ \vdots \\ a_K \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial J_e(\mathbf{a})}{\partial a_0} \\ \vdots \\ \frac{\partial J_e(\mathbf{a})}{\partial a_K} \end{bmatrix} \\
 &= \begin{bmatrix} a_0 \\ \vdots \\ a_K \end{bmatrix} + \alpha \frac{2}{N} \begin{bmatrix} x_0(0) & x_0(1) & \cdots & x_0(N-1) \\ \vdots & \vdots & & \vdots \\ x_K(0) & x_K(1) & \cdots & x_K(N-1) \end{bmatrix} \left(\begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(N-1) \end{bmatrix} - \begin{bmatrix} \hat{y}(0) \\ \hat{y}(1) \\ \vdots \\ \hat{y}(N-1) \end{bmatrix} \right) \\
 &= \begin{bmatrix} a_0 \\ \vdots \\ a_K \end{bmatrix} + \alpha \frac{2}{N} \left\{ \begin{bmatrix} d(0)x_0(0) + d(1)x_0(1) + \cdots + d(N-1)x_0(N-1) \\ \vdots \\ d(0)x_K(0) + d(1)x_K(1) + \cdots + d(N-1)x_K(N-1) \end{bmatrix} \right\}.
 \end{aligned}$$

Anexo VI:

Efeitos de outliers no treinamento de
modelos de ML

Impacto de *outliers* no modelo aprendido



Função original: $y = 1 + x + 2x^2$

- **Outliers** podem levar o modelo a se concentrar demais em reduzir os erros grandes, em detrimento de um bom ajuste geral.
- A consequência disso é a **distorção do modelo aprendido**, fazendo com que ele se **distancie** de um **mapeamento que represente o comportamento dos dados típicos**.

Figuras

